

第9章 图形用户界面设计

(Chapter 9 Graphical User Interface Design)

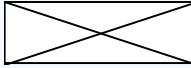
9.1 图形用户界面编程 (GUI Programming)

9.1.1 图形对象 (Graphic Object)

9.1.2 菜单设计 (Menu Design)

9.1.3 界面设计 (Interfacial Design)

9.1.4 标准对话框及其调用 (Standard Dialog Boxes and Calls)



第9章 图形用户界面设计

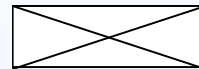
(Chapter 9 Graphical User Interface Design)

9.2 图形用户界面开发环境 (GUIDE)

9.2.1 GUIDE窗口 (Window)

9.2.2 GUIDE常用工具 (Common Tools)

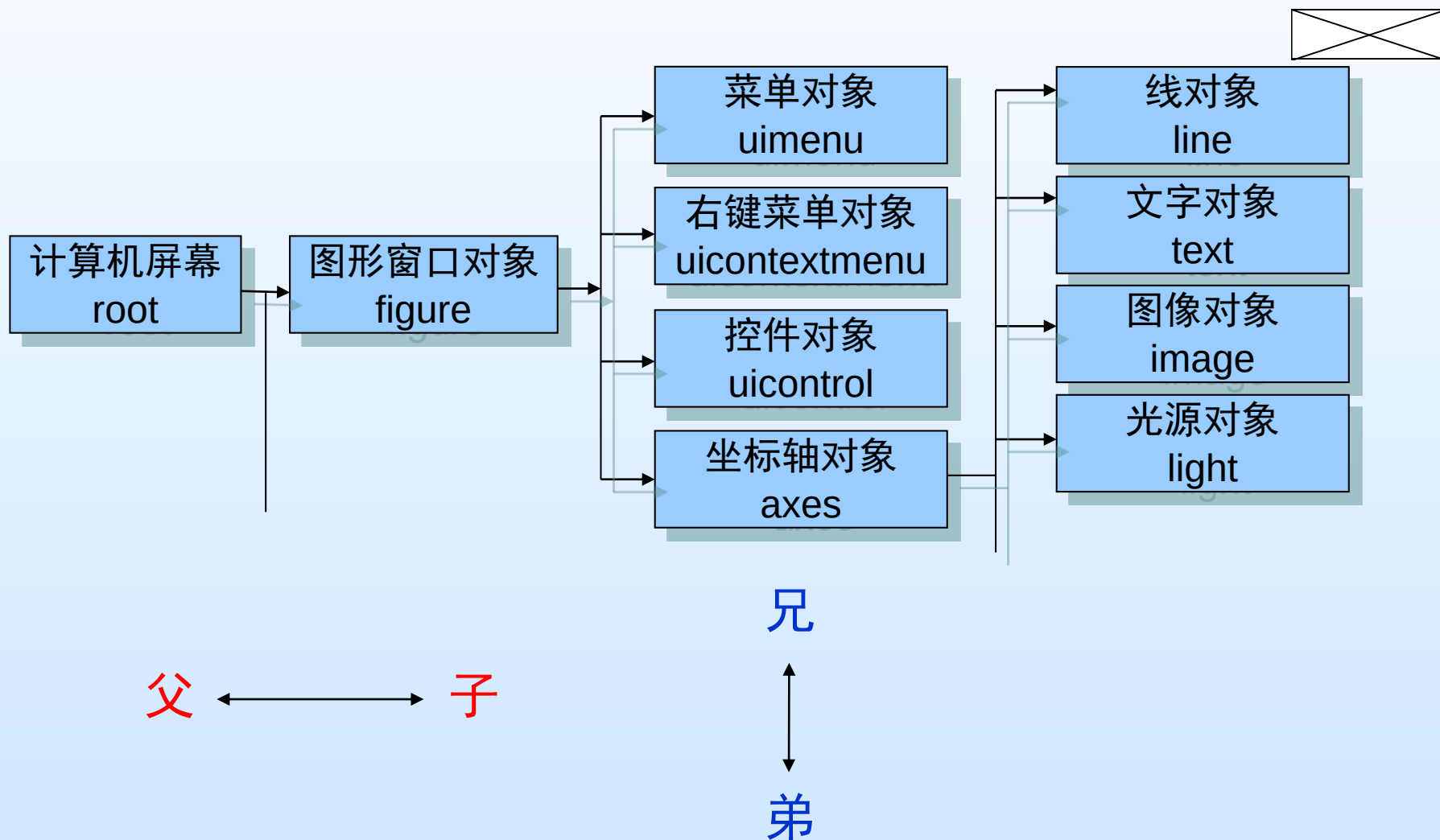
9.2.3 GUIDE应用示例 (Application Example)



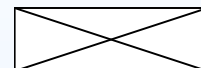
9.1 图形用户界面编程 (GUI Programming)

用户界面是用户与计算机进行信息交流的方式。图形用户界面（或GUI）是包含图形对象，如：窗口、图标、菜单和文本的用户界面。

MATLAB把构成图形的各个基本要素称为**图形对象**。这些对象包括计算机屏幕、图形窗口(Figure)、坐标轴(Axes)、用户菜单(Uimenu)、用户控件(Uicontrol)、曲线(Line)、曲面(Surface)、文字(Text)、图像(image)、光源(Light)、区域块(Patch)和方框(Rectangle)。系统将每一个对象按树型结构组织起来。每个图形对象都可以被独立地操作。如下图所示：



MATLAB图形界面中各对象关系
(Handle graphics system diagram)



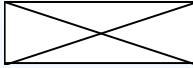
9.1.1 图形对象（Graphic Object）

1. 图形对象句柄（Graphic object handle）

MATLAB在创建每一个图形对象时，都为该对象分配唯一的一个值，称其为**图形对象句柄**(Handle)。句柄是图形对象的唯一标识符。计算机屏幕作为根对象由系统自动建立，其句柄值为0

MATLAB提供了若干个用于获取已有图形对象句柄的函数。常用的如下表：

函数	功能
gcf	获取当前图形窗口的句柄(get current figure)
gca	获取当前坐标轴的句柄(get current axis)
gco	获取最近被单击的图形对象的句柄(get current object)
gcbo	获得当前正在执行调用的对象的句柄
gcbf	获取包括正在执行调用的对象的图形句柄
delete	删除句柄所对应的图形对象
findobj	获取具有指定的属性值的对象的句柄

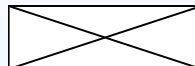


Example 9-1 绘制正弦曲线并查看有关对象的句柄

程序：

```
x=0:pi/100:2*pi;  
y=sin(x);  
h0=plot(x,y,'r')    %曲线对象句柄  
h1=gcf              %图形窗口句柄  
h2=gca              %坐标轴句柄  
h3=findobj(gca,'Color','r')
```

在获取对象的句柄后，可以通过句柄来设置或获取对象的属性。



h0 =

[Line](#) with properties:

```

        Color: [1 0 0]
        LineStyle: '-'
        LineWidth: 0.5000
        Marker: 'none'
        MarkerSize: 6
        MarkerFaceColor: 'none'
        XData: [1x201 double]
        YData: [1x201 double]
        ZData: [1x0 double]

```

Show [all properties](#)

h1 =

[Figure](#) (1) with properties:

```

        Number: 1
        Name: ''
        Color: [0.9400 0.9400 0.9400]
        Position: [488 342 560 420]
        Units: 'pixels'

```

Show [all properties](#)

h2 =

[Axes](#) with properties:

```

        XLim: [0 7]
        YLim: [-1 1]
        XScale: 'linear'
        YScale: 'linear'
        GridLineStyle: '-'
        Position: [0.1300 0.1100 0.7750 0.8150]
        Units: 'normalized'

```

Show [all properties](#)

h3 =

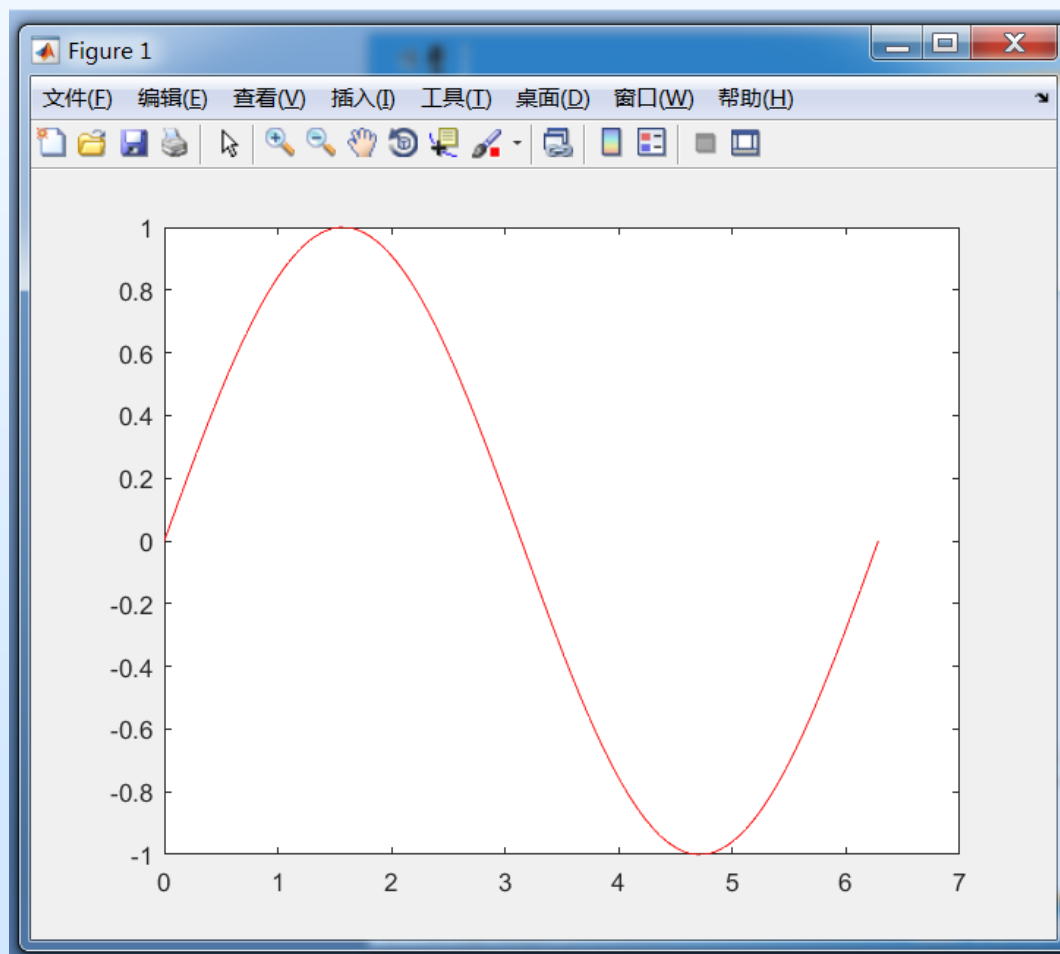
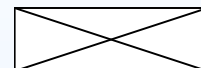
[Line](#) with properties:

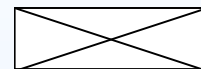
```

        Color: [1 0 0]
        LineStyle: '-'
        LineWidth: 0.5000
        Marker: 'none'
        MarkerSize: 6
        MarkerFaceColor: 'none'
        XData: [1x201 double]
        YData: [1x201 double]
        ZData: [1x0 double]

```

Show [all properties](#)





2. 图形对象属性及其设定 (Graphic object properties and their Settings)

(1) 属性名与属性值 (Attribute name and attribute value)

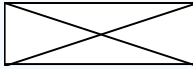
MATLAB给每种对象的每一个属性规定了一个名字，称为属性名，而属性名的取值称为属性值。它们在创建或修改属性的指令中总是成对出现。

例如，**LineStyle**是曲线对象的一个属性名，它的值决定着线型，取值可以是'-'、':'、'-.'、'--'或'none'。

在属性名的写法中，不区分字母的大小写，而且在不引起歧义的前提下，属性名不必写全。

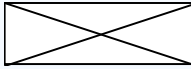
例如**lines**就代表**LineStyle**。

此外属性名要用单引号括起来。



(2) 图形对象的公共属性 (Public properties of graphics objects)

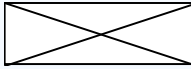
- 1) Children属性——该对象的所有子对象的句柄组成的一个向量。
- 2) Parent 属性——该对象的父对象的句柄。
- 3) Tag属性——给对象定义的一个标识符，以便引用。
- 4) Type属性——表示该对象的类型，属性的值不可改变。
- 5) Visible属性——该属性的取值是on(缺省值)或off，决定对象是否显示。
- 6) ButtonDownFcn属性、CreateFcn属性、DeleteFcn属性——均为字符型，取值为某M文件名或一小段程序，分别用于单击对象、创建对象时与取消对象时加以执行。



7) UserData属性——该属性的取值为一矩阵（缺省空），可以将一个对象有关的比较重要的数据存储在其中，借以达到传送数据的目的。

8) Color属性——既可以是MATLAB预定义的几种颜色（用字符表示，如'r'代表红色），也由红绿蓝三原色组成的向量，取值范围为0到1。常见的如下表：

颜色	黄	紫	青	红	绿	蓝	白	黑
字符形式	y	m	c	r	g	b	w	k
向量形式	[1 1 0]	[1 0 1]	[0 1 1]	[1 0 0]	[0 1 0]	[0 0 1]	[1 1 1]	[0 0 0]



(3) 属性的设置 (Setting of properties)

当创建一个对象时，必须给对象的各种属性赋予必要的属性值，否则系统使用默认值。用户可通过set函数重新设置对象的属性值，也可以通过get函数获取这些属性值。

set函数的调用格式为：

set(句柄,属性名1,属性值1,属性名2,属性值2,...)

其中句柄用于指明要操作的图形对象。如果省略所有的属性名与属性值，则将显示句柄所有的允许属性。

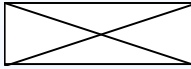
如：**set(h0,'color','b','LineWidth',3,'LineStyle','--')**

(将**Example 9-1**绘制的曲线颜色设置成蓝色，线宽为3点，线型为破折号)

p_vect=[100,200,300,200] %[left, bottom, width, height]

set(h1, 'Position' ,p_vect)

(将**Example 9-1**窗口位置设为向量p_vect所指定的值)



get函数的调用格式为:

V=get(句柄[,属性名])

其中V是返回的属性值。如果省略属性名, 则将返回句柄所有的属性值。

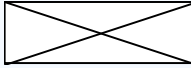
如: **c=get(h0,'color')** %**get(h1)**返回窗口的所有属性值
返回具有句柄h0对象的颜色

而下列形式的get函数可获取屏幕的分辨率:

V=get(0,'screensize')

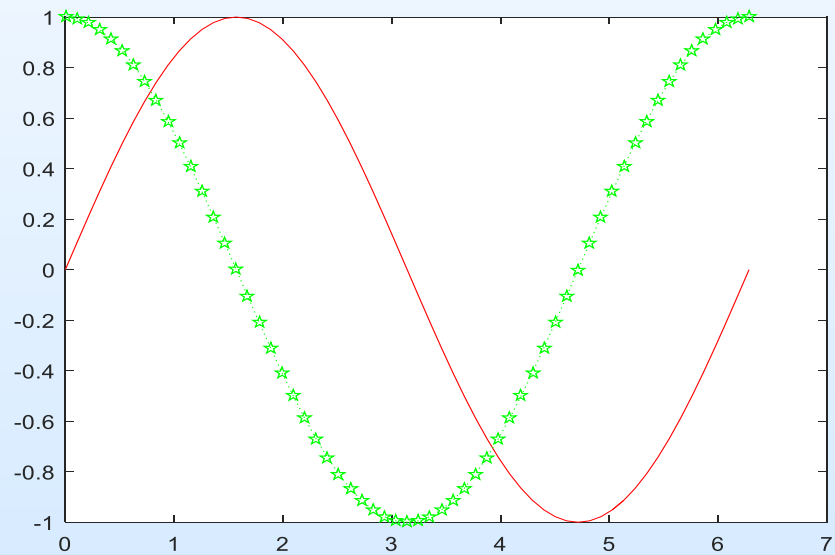
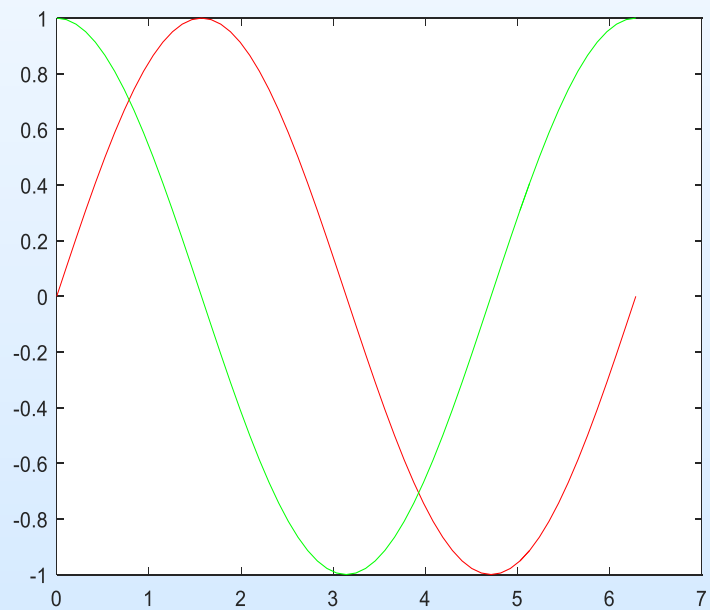
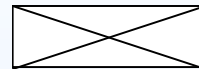
此时get函数将返回一个 1×4 的向量V, 前两个代表屏幕左下角的坐标(1, 1), 而后两个分别代表屏幕的宽度与高度, 即屏幕分辨率。如要建立一个起始与屏幕左下角, 高度与宽度分别是现行分辨率大小一半的图形窗口, 可用命令:

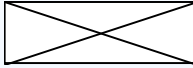
figure('position',[1,1,0.5*V(3),0.5*V(4)])



Example 9-2 在同一坐标下绘制红、绿两根不同曲线，希望获得绿色曲线的句柄，并对其线型重新设置

```
x=0:pi/30:2*pi;
y=sin(x);
z=cos(x);
plot(x,y,'r',x,z,'g');
h=get(gca,'Children'); %获取两曲线的句柄向量
for k=1:size(h)
    if get(h(k),'color')==[0 1 0]
        h1=h(k);
        break;
    end
end
pause;
set(h1,'Linestyle',':','Marker','p'); %重新设置绿色曲线
```





3. 图形对象的创建 (Creation of graphic objects)

(1) 图形窗口对象 (Graphic window object)

图形窗口是MATLAB中很重要的一类图形对象。MATLAB中的一切图形图像的输出都是在图形窗口中完成的。掌握好图形窗口的控制方法，对于充分发挥MATLAB的图形功能和设计高质量的用户界面是十分重要的。

建立图形窗口对象使用figure函数，其调用格式为：

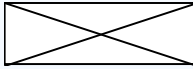
H=figure(属性名1,属性值1,属性名2,属性值2,...)

MATLAB通过对属性的操作来改变图形窗口的形式。也可使用：**figure** 或 **H=figure**

按MATLAB缺省的属性值建立图形窗口。还可以通过：

figure(窗口句柄)

调用来显示指定窗口或将之设定为当前窗口。要关闭图形窗口使用：**close(H)** %**close all**关闭所有的图形窗口



MATLAB为每个图形窗口提供了很多属性。这些属性及其取值控制着图形窗口对象。除前面的公共属性外，常用的还有：

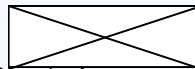
1) **Name属性**——取值可以是任何字符串，它的缺省值为空。这个字符串作为图形窗口的标题。一般情况下，其标题形式为：Figure No.1:字符串。

2) **NumberTitle属性**——取值是on(缺省值)或off。决定着在图形窗口的标题中是否以“Figure No.n:”为标题前缀，这里n是图形窗口的序号，即句柄值。

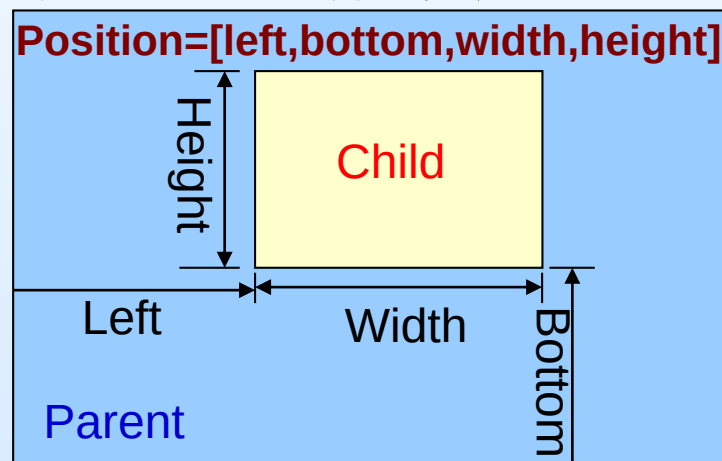
3) **MenuBar属性**——取值可以是figure(缺省值)或none。用来控制图形窗口是否应该具有菜单条。如果它的属性值为none，则表示该图形窗口没有菜单条。

4) **Resize属性**——确定是否可以改变图形窗口的大小。

5) **ToolBar属性**——是否显示工具条。

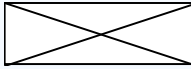


6) **Position**属性——设定对象的位置和大小，为 1×4 向量，前两个值为窗口左下角横纵坐标值，后两个元素值为窗口的宽度和高度，其单位由**units**属性设定。



7) **units**属性——取值是下列字符串中的一种：pixels(像素，默认)、points(磅)、inches(英寸)、centimeters(厘米) 和 normalized(相对单位)。

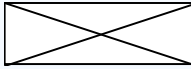
该属性定义图形窗口使用的长度单位，从而决定窗口的大小与位置。除normalized以外，其它单位都是绝对度量单位。相对单位normalized将屏幕左下角对应为(0,0)，而右上角对应为(1.0,1.0)。该属性将影响一切定义的属性项。



8) **KeyPressFcn**属性、**WindowButtonDownFcn**属性、**WindowButtonMotionFcn**属性、**WindowButtonUpFcn**属性——均为字符型，取值为某M文件名或一小段程序，分别用于键盘按下、鼠标按下、鼠标移动、鼠标释放时加以执行。

9) **Pointer**属性——设置鼠标的显示形式。取值及意义如下表：

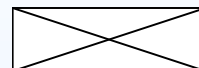
选项	意义	选项	意义	选项	意义
crosshair		arrow	 (缺省)	watch	
topl		topr		botl	
botr		circle		cross	
fleur		custom	自定义		

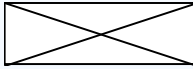


Example 9-3 建立一个图形窗口。该图形窗口没有菜单条，标题名称为“图形窗口示例”，起始于屏幕左下角、宽度和高度分别为300像素点和150像素点，背景颜色为蓝色，且当用户单击鼠标时，将显示“瞧你,当我不知道！”字样。

命令如下：

```
hf=figure('Color',[0,0,1],'Position',[1,1,300,150],...  
    'Name','图形窗口示例',...  
    'NumberTitle','off',...  
    'MenuBar','none',...  
    'ButtonDownFcn','disp('瞧你,当我不知道！ ');');%自  
动执行属性ButtonDownFcn对应的属性值。
```

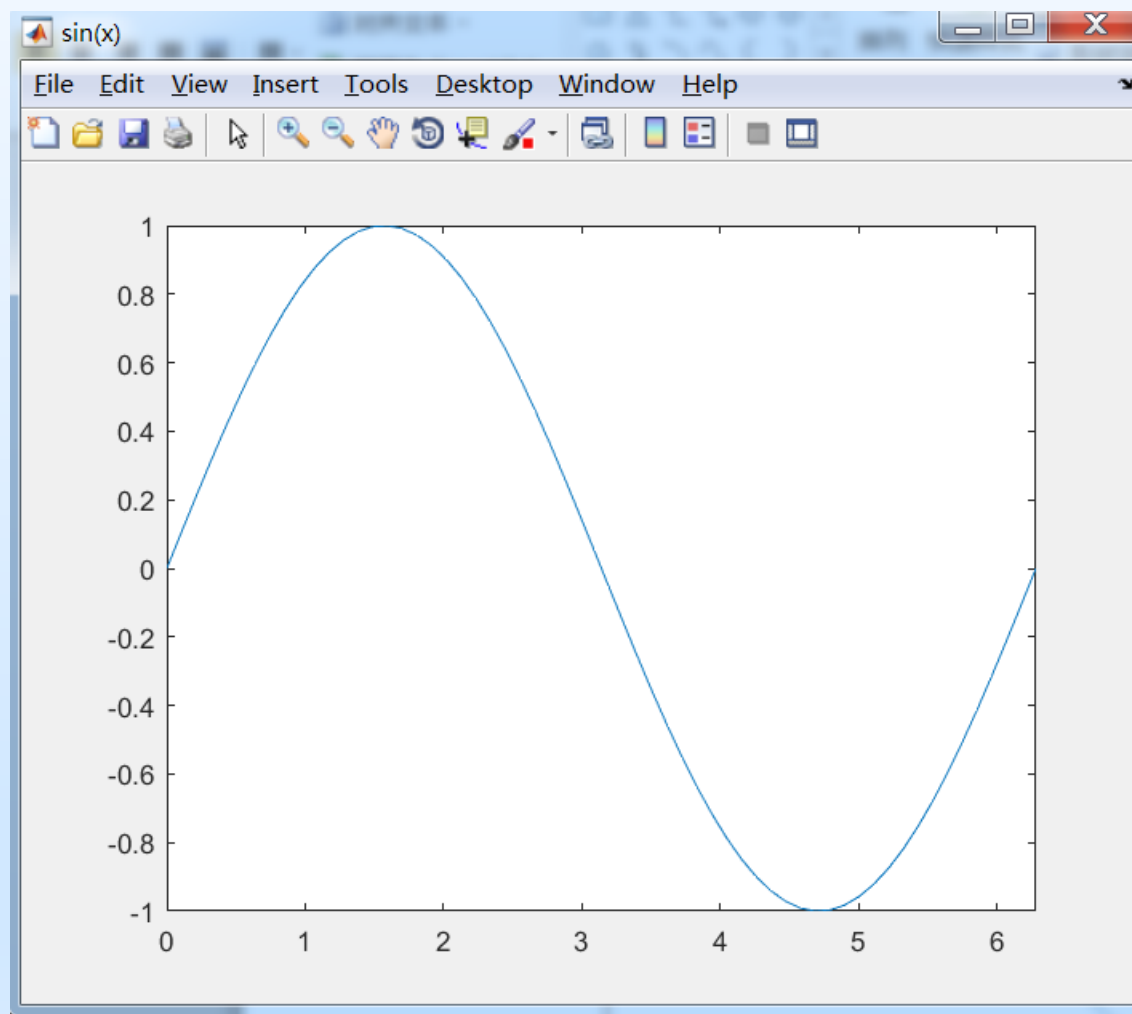
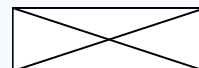


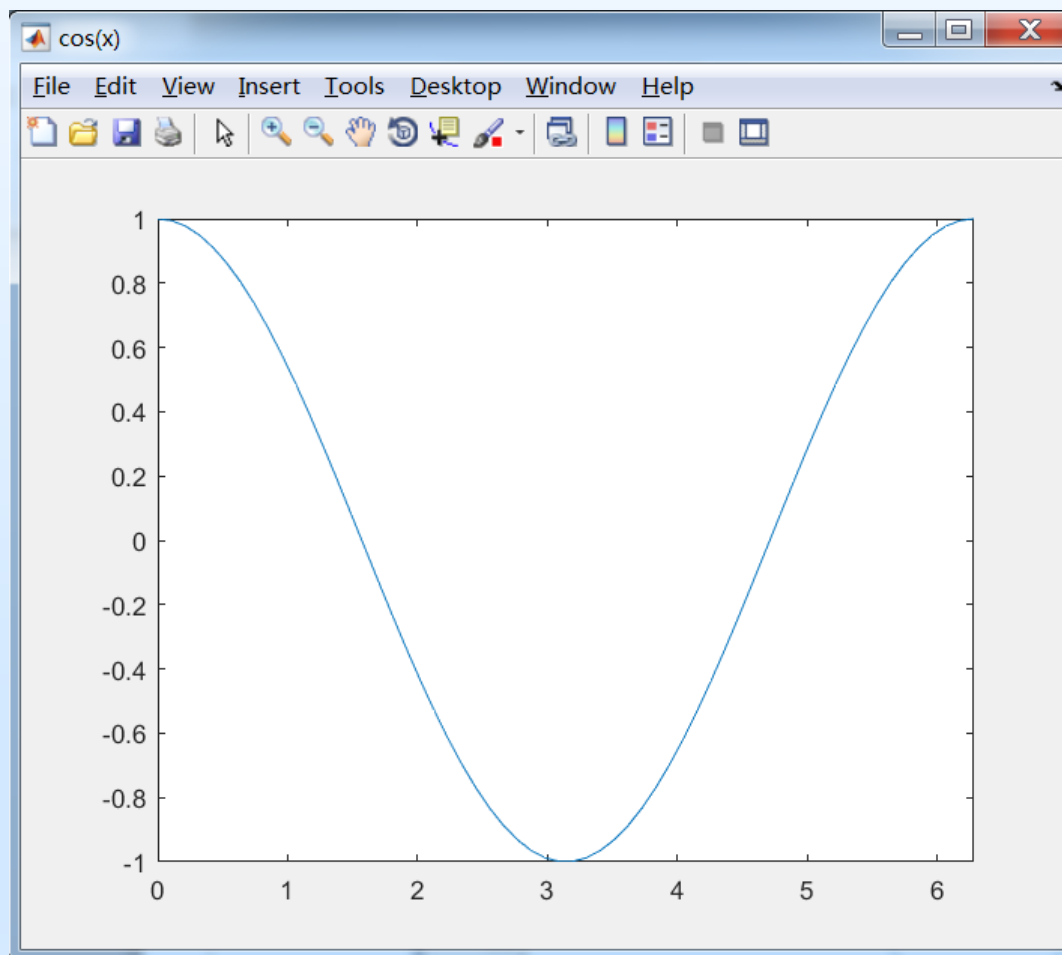
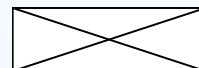


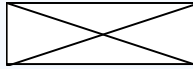
Example 9-4 分别在2个不同的图形窗口绘制出正弦、余弦曲线。要求先建立正弦曲线窗口并绘图，关闭后再建立余弦曲线窗口并绘图。

程序如下：

```
x=linspace(0,2*pi,60);  
y=sin(x);  
z=cos(x);  
%命令待用  
C=['figure("Name","cos(x)","NumberTitle","off");'...  
    'plot(x,z);axis([0,2*pi,-1,1]);'];  
%先创建第1个图形窗口并绘制曲线  
figure('Name','sin(x)','DeleteFcn',C,'NumberTitle','off');...  
    plot(x,y);axis([0,2*pi,-1,1]);
```







(2) 坐标轴对象 (**Coordinate object**)

建立坐标轴对象使用axes函数，其调用格式为：

H=axes(属性名1,属性值1,属性名2,属性值2,...)

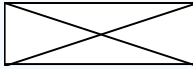
调用axes函数用指定的属性在当前图形窗口创建坐标轴，并将其句柄赋给左边的句柄变量H。

使用axes 函数按MATLAB缺省的属性值在当前图形窗口创建坐标轴：

axes 或 H = axes

用axes 函数建立坐标轴之后，还可以调用axes 函数将之设定为当前坐标轴，且坐标轴所在的图形窗口自动成为当前图形窗口：

axes(H) %这里引用的H必须存在



MATLAB为每个坐标轴对象提供了很多属性。除公共属性外，常见坐标轴对象属性。

1) **Box属性**——取值是on(缺省值)或off。决定着在坐标轴是否带边框。

2) **GridLineStyle属性**——取值可以是默认值、‘-’、‘-.’、‘--’或‘none’，它的值决定着网格线的类型。

3) **Position属性**——是一个4元素的向量，决定坐标轴的位置。

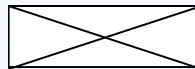
4) **Units属性**——度量单位，同前

5) **Title属性**——该属性的取值是坐标轴标题文字对象的句柄，可以对坐标轴标题文字进行操作。如：

```
h=get(gca,'Title');
```

```
set(h,'Color','b'); %设置标题颜色
```

6) **Xlabel、Ylabel、Zlabel属性**——决定坐标轴说明文字的句柄，同Title属性。

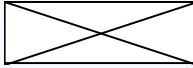


7) **XLim、YLim、ZLim属性**——定义各坐标轴的上、下限，默认值为[0,1]，3种属性的取值都是具有两个元素的数值向量。

8) **XScale、YScale、ZScale属性**——定义各坐标轴的刻度类型，3种属性的取值都是linear（默认值）或log。

9) **View属性**——取值是两个元素的数值向量，定义视点方向。

10) **字体属性**——对坐标轴标注的字体进行设置，这类属性有FontName（字体名称）、FontWeight（字形）、FontSize（字体大小单位）、FontAngle（字体角度）等。



Example 9-5 利用坐标轴对象实现图形窗口的任意分割。

程序如下：

```
clf; %清除当前图形窗口的内容，但不关闭窗口。
```

```
x=linspace(0,2*pi,20);
```

```
y=sin(x);
```

```
axes('Posi',[0.2,0.2,0.2,0.7]);
```

```
plot(x,y);title('sin(x)-1');
```

```
axes('Posi',[0.4,0.5,0.2,0.1]);
```

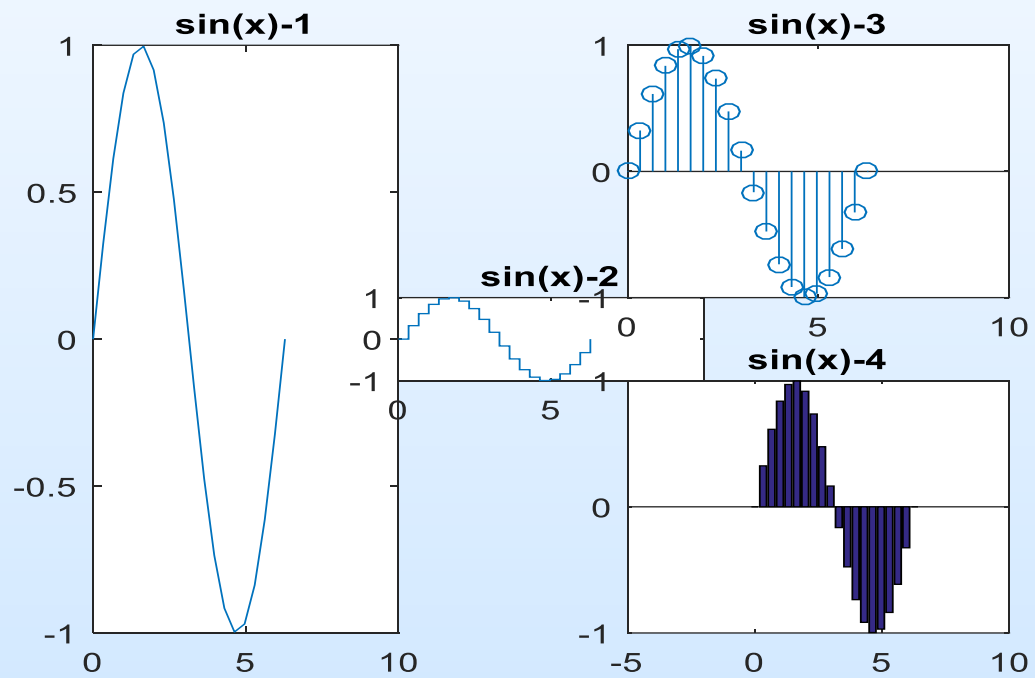
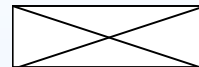
```
stairs(x,y);title('sin(x)-2');
```

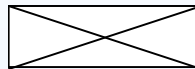
```
axes('Posi',[0.55,0.6,0.25,0.3]);
```

```
stem(x,y);title('sin(x)-3');
```

```
axes('Posi',[0.55,0.2,0.25,0.3]);
```

```
bar(x,y);title('sin(x)-4');
```





(3) 曲线对象 (Curve object)

建立曲线对象使用line函数，调用它的命令形式为：

H=line(x,y,z,属性名1,属性值1,属性名2,属性值2,...)

其中，对x，y，z的解释与高层曲线函数plot和plot3等一样，其余的解释与前面介绍过的figure和axes函数类似。

每个曲线对象也具有很多属性。除公共属性外，其他常用属性如下：1) **Color属性**——定义曲线的颜色，取值是代表颜色的字符或RGB值。

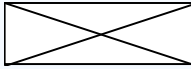
2) **LineStyle属性**——定义线型

3) **LineWidth属性**——定义线宽，默认值为0.5磅

4) **Marker属性**——定义数据点标记符，默认为none

5) **MarkerSize属性**——定义数据点标记符的大小，默认值为6磅

6) **XData、YData、ZData属性**——取值是数值向量或矩阵，分别代表曲线对象的3个坐标轴数据。



Example 9-6 利用曲线对象绘制曲线

程序:

```
t=0:pi/10:2*pi;
```

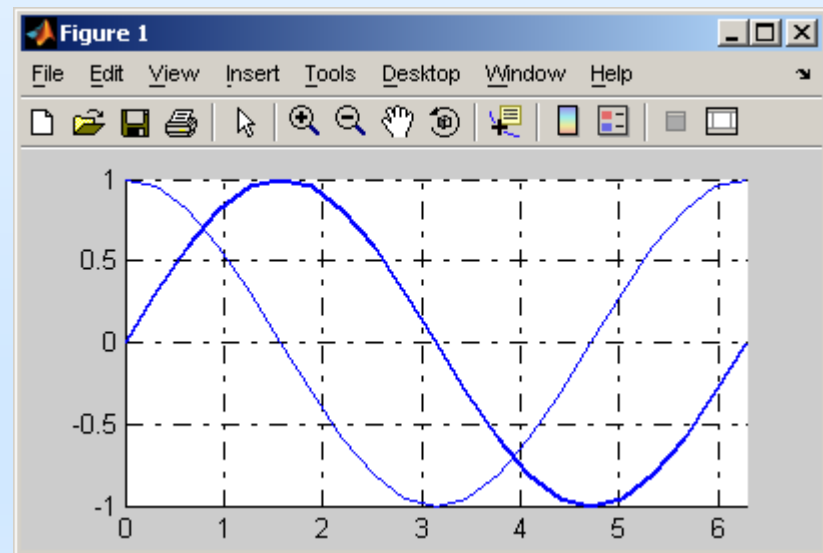
```
h=figure('Position',[30,100,400,200]);
```

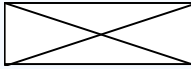
```
axes('GridLineStyle','-.','XLim',[0,2*pi],'YLim',[-1,1]);
```

```
line(t,sin(t),'LineWidth',2);
```

```
line(t,cos(t));
```

```
grid on;
```





(4) 文字对象 (Text object)

使用text函数可以根据指定位置和属性值添加文字说明，并保存句柄。

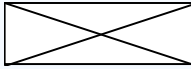
调用该函数的命令形式为：

$H = \text{text}(x, y, z, \text{'说明文字'}, \text{属性名1}, \text{属性值1}, \text{属性名2}, \text{属性值2}, \dots)$

其中，说明文字中除使用标准的ASCII字符外，还可使用LATEX格式的控制字符。

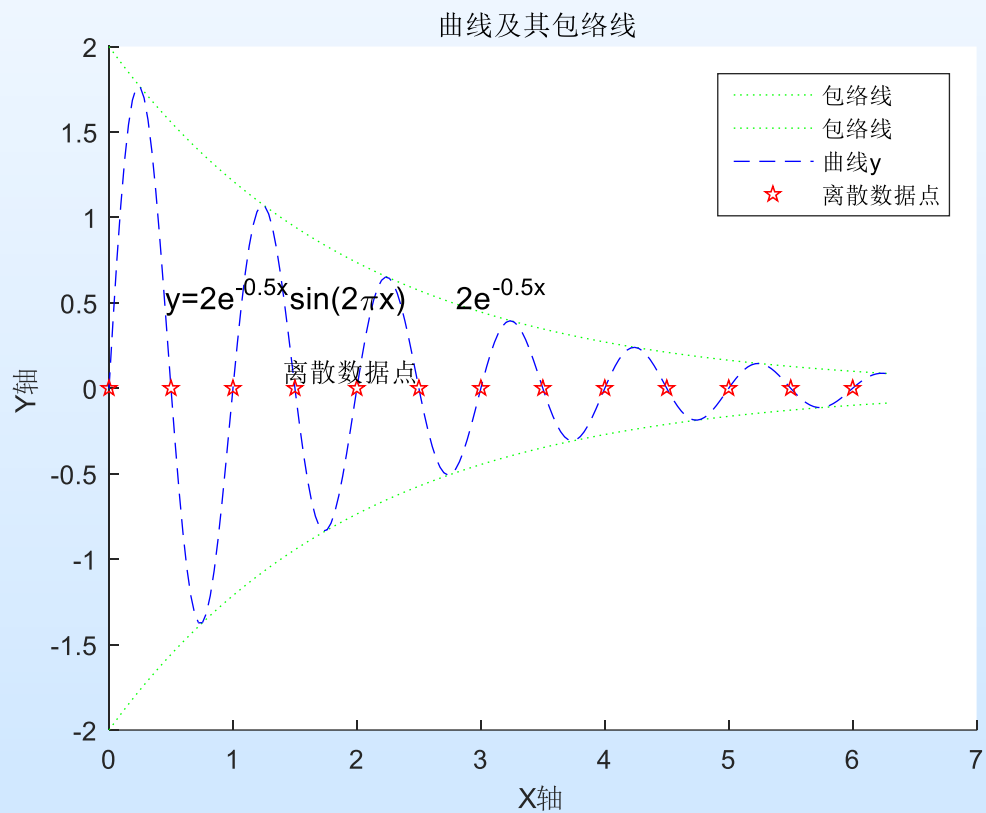
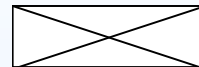
除公共属性外，文字对象其他常用属性如下：

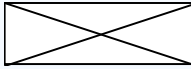
- 1) **String属性**——记录文字标注的内容，字符串（矩阵）
- 2) **Interpreter属性**——控制对文字的解释（Latex或ASCII）
- 3) **Rotation属性**——文字旋转角度，默认值为0
- 4) **BackgroundColor、EdgeColor属性**——背景或边框颜色
- 5) **HorizontalAlignment属性**——对齐方式，默认left、center
- 6) **字体属性**——如FontName、FontSize等



Example 9-7 利用曲线对象绘制曲线并利用文字对象完成标注。
程序如下：

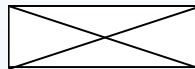
```
x=(0:pi/100:2*pi)';  
y1=2*exp(-0.5*x)*[1,-1];  
y2=2*exp(-0.5*x).*sin(2*pi*x);  
x1=(0:12)/2;  
y3=2*exp(-0.5*x1).*sin(2*pi*x1);  
line(x,y1,'LineStyle',':','color','g');  
line(x,y2,'LineStyle','--','color','b');  
line(x1,y3,'LineStyle','none','Marker','p','color','r');  
title('曲线及其包络线');xlabel('X轴');ylabel('Y轴');  
text(2.8,0.55,'2e^{-0.5x}', 'FontSize',12);  
text(0.45,0.55,'y=2e^{-0.5x}sin(2{\pi}x)','FontSize',12);  
text(1.4,0.1,'离散数据点');  
legend('包络线','包络线','曲线y','离散数据点')
```





Example 4-5 给图形添加图形标注。

```
x=(0:pi/5:2*pi)';  
y1=2*exp(-0.5*x)*[1,-1];  
y2=2*exp(-0.5*x).*sin(2*pi*x);  
x1=(0:12)/2;  
y3=2*exp(-0.5*x1).*sin(2*pi*x1);  
plot(x,y1,'g:',x,y2,'b--',x1,y3,'rp');  
title('曲线及其包络线');      %加图形标题  
xlabel('Variable X');          %加X轴说明  
ylabel('Variable Y');          %加Y轴说明  
text(2.8,0.5, '包络线');      %在指定位置添加图形说明  
text(0.5,0.5,'曲线y');  
text(1.4,0.1,'离散数据点');  
legend('包络线','包络线','曲线y','离散数据点') %加图例
```



(5) 曲面对象 (Surface object)

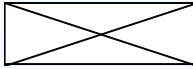
建立曲面对象使用surface函数，调用它的命令形式为：

`H=surface(x,y,z,属性名1,属性值1,属性名2,属性值2,...)`

其中对x,y,z的解释与高层曲面函数mesh和surf等一样，其余的解释与前面介绍过的figure和axes等函数类似。

每个曲面对象也具有很多属性。除公共属性外，其他常用属性如下：

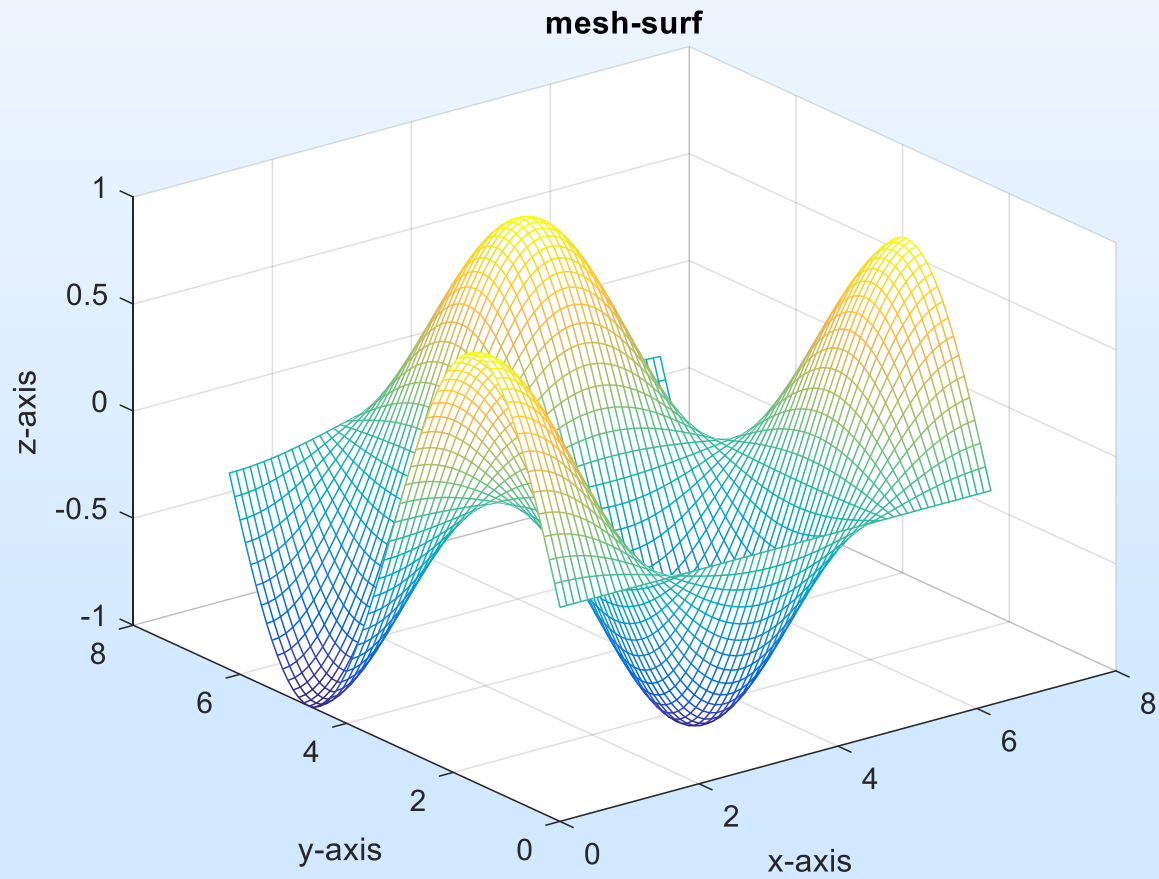
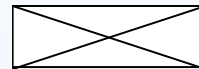
- 1) EdgeColor属性——曲面网格线的颜色或着色方式，还可以是flat、interp或none，默认黑色
- 2) FaceColor属性——曲面网格片的颜色或着色方式
- 3) LineStyle、LineWidth属性——曲面网格线的线型或线宽，默认0.5磅
- 4) Marker、MarkerSize属性——数据点标记符号或大小（默认6磅）
- 5) XData、YData、ZData属性——取值是数值向量或矩阵，分别代表曲面对象的3个坐标轴数据。

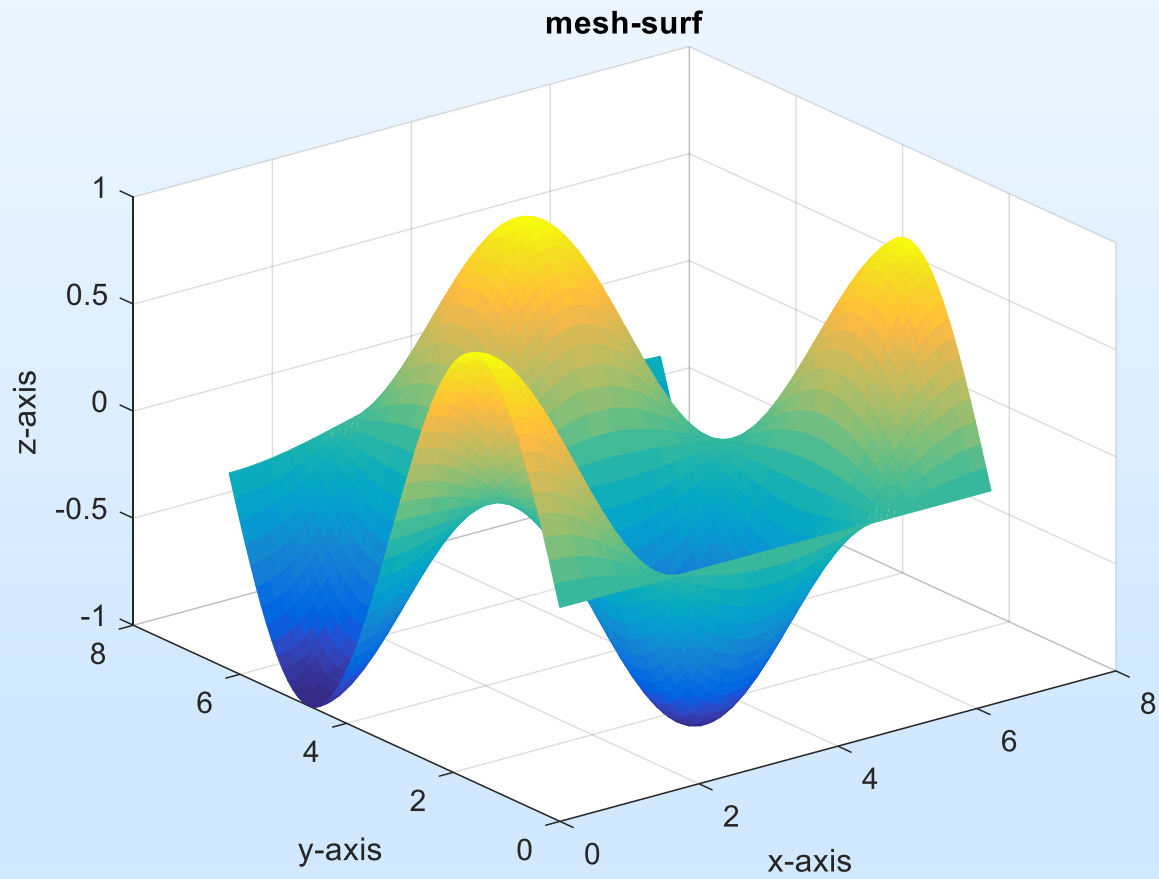
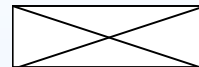


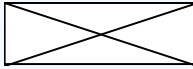
Example 9-8 利用曲面对象绘制三维曲面 $z=\sin(y)\cos(x)$ 。

程序如下：

```
x=0:0.1:2*pi;  
[x,y]=meshgrid(x);  
z=sin(y).*cos(x);  
axes('view',[-37.5,30]);  
hs=surface(x,y,z,'FaceColor','w','EdgeColor','flat');  
grid on;  
xlabel('x-axis'),ylabel('y-axis'),zlabel('z-axis');  
title('mesh-surf');  
pause;  
set(hs,'FaceColor','flat');
```







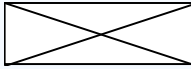
(6) 方框对象 (**Box object**)

在MATLAB中，矩形、圆角矩形、椭圆、圆及对应的区域图形都用方框对象表示。使用rectangle函数可以创建方框对象，其调用格式为：

H= rectangle (属性名1,属性值1,属性名2,属性值2,...)

除公共属性外，其他常用属性如下：

- 1) EdgeColor属性——边框线的颜色
- 2) LineStyle、LineWidth属性——定义方框的线型或线宽，默认0.5磅
- 3) Curvature属性——定义方框边的曲率
- 4) Position属性——相对坐标轴原点定义的方框位置。



Example 9-9 在同一个图形窗口中创建矩形、圆角矩形、椭圆和圆各一个，并使用了不同的线型和线宽。

程序如下：

```
subplot(2,2,1);
```

```
rectangle('Position',[2,3,25,15],'LineWidth',3);
```

```
subplot(2,2,2);
```

```
rectangle('Position',[2,3,15,10], 'Curvature',0.4,'LineWidth',2);
```

```
subplot(2,2,3);
```

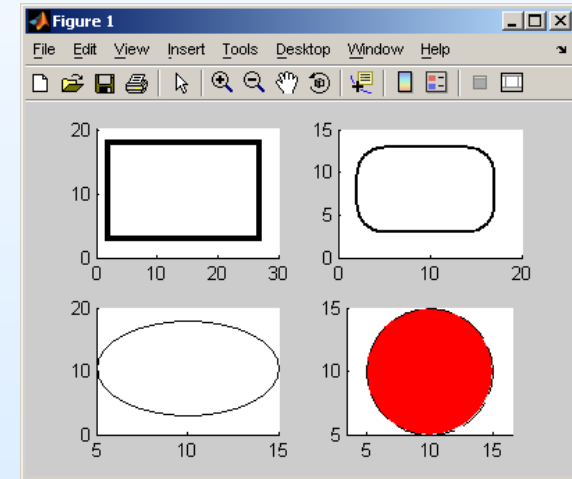
```
rectangle('Position',[5,3,10,15],'Curvature',[1,1]);
```

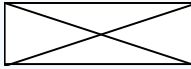
```
subplot(2,2,4);
```

```
rectangle('Position',[5,5,20,20],'Curvature',[1,1],...
```

```
'LineStyle','--','FaceColor','r');
```

```
axis equal
```



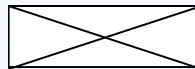


Experiment Content

Ex-1: 简要描述下面程序最终的运行图形。

```
x=2*pi:pi/40:2*pi;  
y=sin(x); z=sin(x);  
H1=plot(x,y,x,z);  
set(H1(1), color, [1,0.5,0], 'linewidth', 3)  
set(H1(2),'color', [0.75,0.75,1])  
title('Handle Graphic Example')  
H1_text=get(gca,'Title')  
set(H1_text, 'FontSize', 15)
```

Ex-2: 建立一个图形窗口，使之背景颜色为红色，并在窗口上保留原有的菜单项，而且在按下左键之后显示出Left Button Pressed字样。



Ex-3: 以任意位置子图形式绘制出正弦、余弦、正切和余切函数曲线。

Ex-4: 利用图形对象绘制下列曲线，要求先利用默认属性绘制曲线，然后通过图形句柄操作来改变曲线的颜色、线型和线宽，并利用文本对象给曲线添加文字标注。

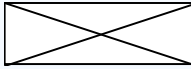
$$(1) \quad y = \frac{1}{2} \ln(x + \sqrt{1 + x^2})$$

$$(2) \quad \begin{cases} x = t^2 \\ y = 5t^3 \end{cases}$$

Ex-5: 利用图形对象绘制下列三维图形，要求对图形进行光照处理。

$$(1) \quad z = x^2 + y^2 - 5\sin(xy)$$

$$(2) \quad z = y^3$$



9.1.2 菜单设计（ Menu Design ）

1. 建立用户菜单（ Create user menu ）

要建立用户菜单可用uimenu函数，因其调用方法不同，该函数可以用于建立一级菜单项和子菜单项。其调用格式为：

$H = \text{uimenu}(mH, \text{属性名1}, \text{属性值1}, \text{属性名2}, \text{属性值2}, \dots)$

说明：

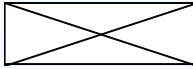
如mH为窗口句柄，则H为一级菜单句柄，

如mH为一级菜单项句柄，则H为二级菜单句柄，

2. 菜单对象常用属性（ Common properties of menu objects ）

1) **Accelerator**属性——指定菜单项等价的按键或快捷键。

2) **BackgroundColor**属性——uimenu背景色,是一个3元素的RGB向量或MATLAB预先定义的颜色名称。缺省的背景色是亮灰色



3) **Callback**属性——MATLAB回调字符串，选择菜单项时，回调串传给函数eval作为对菜单的响应；初始值为空矩阵

4) **Checked**属性——被选项的校验标记

on: 校验标记出现在所选项的旁边

off（默认值）：校验标记不显示

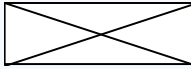
5) **Enable**属性——菜单使能状态

on（默认值）：菜单项使能。选择菜单项能将Callback字符串传给eval

off: 菜单项不使能，菜单标志变灰。选择菜单项不起任何作用。

6) **ForegroundColor**属性——菜单前景（文本）色,是一个三元素的RGB向量或MATLAB预先定义的颜色名称。缺省的前景色是黑色

7) **Label**属性——用于定义菜单项的名字，含有菜单项标志的文本串。字符前面有&定义热键，它由Alt-字符激活



8) **Position**属性——菜单对象的相对位置。顶层菜单从左到右编号，子菜单从上至下编号

9) **Separator**属性——分割符-线模式

on: 分割线在菜单项之上

off (默认): 不画分割线

10) **Visible**属性——菜单对象的可视性

on (默认): 菜单对象在屏幕上可见

off: 菜单对象不可见

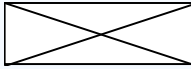
11) **ButtonDownFcn**属性——当对象被选择时，MATLAB的回调串传给函数eval。初始值为空矩阵。

12) **Children**属性——其它菜单对象的句柄。

13) **Interruptible**属性——指明ButtonDownFcn和CallBack串可否中断

no (默认): 回调不可中断

yes: 回调串可中断



14) Parent属性——父对象的句柄；如果菜单对象是顶层菜单，则为图形对象；若菜单是子菜单，则为父的菜单对象句柄。

15) Tag属性——文本标识串

16) UserData属性——用户指定的数据。可以是矩阵，字符串等等

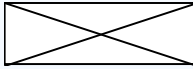
Example 9-10 建立如下图所示的菜单。



程序如下：

```
hh=figure('Name','菜单演示');  
set(hh,'MenuBar','none','NumberTitle','off');  
hm1=uimenu(hh,'Label','文件');  
hm1_1=uimenu(hm1,'Label','新建(&N)','Callback','edit');  
hm1_2=uimenu(hm1,'Label','打开(&O)','Callback','uigetfile');  
hm1_3=uimenu(hm1,'Label','保存(&S)','Callback','');  
hm1_3_1=uimenu(hm1_3,'Label','保存为M文件','Callback','edit');  
hm1_3_2=uimenu(hm1_3,'Label','保存为MAT文件','Callback','');  
hm1_4=uimenu(hm1,'Separator','on','Callback','');  
hm1_5=uimenu(hm1,'Label','退出','Callback','close');  
hm2=uimenu(hh,'Label','颜色');  
hm2_1=uimenu(hm2,'Label','红(&R)','Callback','set(gcf,'Color','red')');  
hm2_2=uimenu(hm2,'Label','蓝(&B)','Callback','set(hh,'Color','blue')');  
hm2_3=uimenu(hm2,'Label','黑(&K)','Callback','set(hh,'Color',[0 0 0])');  
hm3=uimenu(hh,'Label','帮助(&H)');  
hm3_1=uimenu(hm3,'Label','帮助','Callback','help');  
hm3_2=uimenu(hm3,'Label','关于','Callback','close');
```





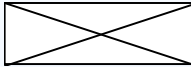
3. 快捷菜单（ shortcut menu ）

快捷菜单是用鼠标右键单击某对象时在屏幕上弹出的菜单。这种菜单出现的位置是不固定的，而且总是和某个图形对象相联系。在MATLAB中，可以使用uicontextmenu函数和图形对象的UIContextMenu属性来建立快捷菜单。

具体步骤为：

- (1) 利用uicontextmenu函数建立快捷菜单。
- (2) 利用uimenu函数为快捷菜单建立菜单项。
- (3) 利用set函数将该快捷菜单和某图形对象联系起来。

Example 9-11 绘制曲线 $y=\sin x/x$ ，并建立一个快捷菜单，用以控制曲线的颜色。



程序如下：

```
x=-3*pi:pi/50:3*pi;
```

```
y=sin(x)./x;
```

```
hline=plot(x,y); %绘制曲线
```

```
cm=uicontextmenu; %创建快捷菜单
```

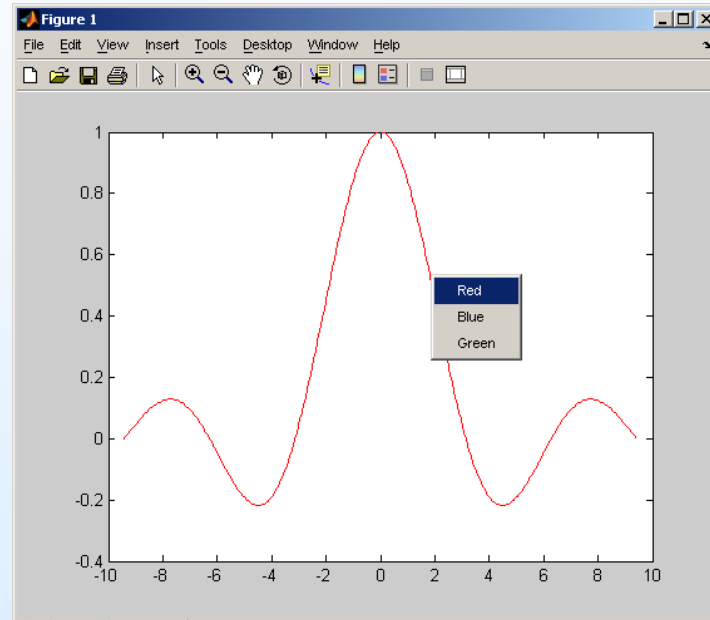
```
%制作具体菜单项，定义相应的回调
```

```
uimenu(cm,'label','Red','callback','set(hline,"color","r")')
```

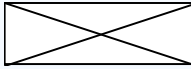
```
uimenu(cm,'label','Blue','callback','set(hline,"color","b")')
```

```
uimenu(cm,'label','Green','callback','set(hline,"color","g")')
```

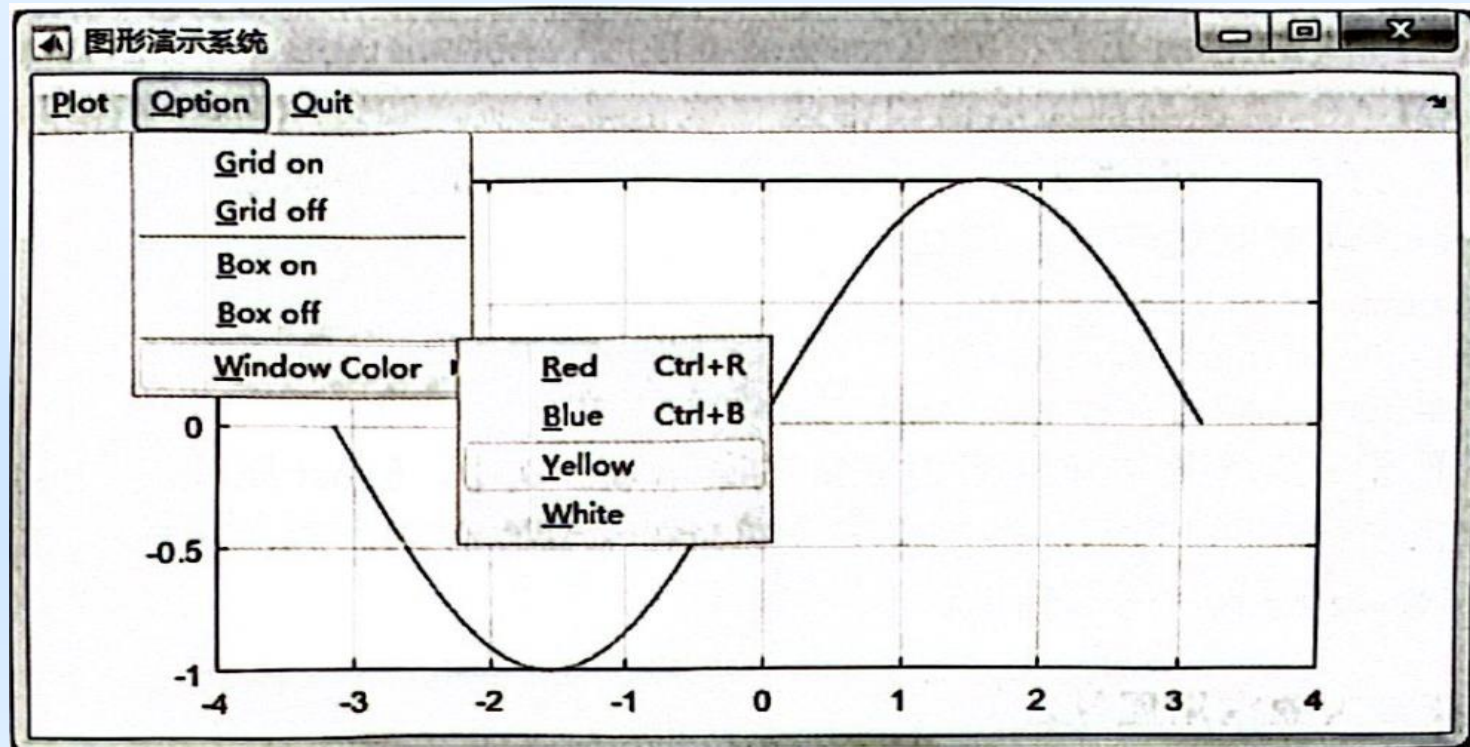
```
set(hline,'UIContextMenu',cm) %使快捷菜单与曲线相联系
```



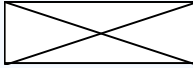
Experiment Content



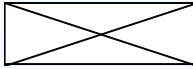
Ex-1: 建立如下图所示的图形演示系统菜单。菜单条中含有3个菜单项：Plot、Option和Quit。Plot中有Sine Wave和Cosine Wave两个子菜单项，分别控制在本图形窗口画出正弦和余弦曲线。Option菜单项的内容如下图所示，其中Grid on和Grid off控制给坐标轴加网格线，Box on和Box off控制给坐标轴加边框，而且这4项只有在画线时才是可选的。Window Color控制图形窗口背景颜色。Quit控制是否退出系统。



Experiment Content



Ex-2: 绘制曲线 $y=2\sin(5x)\sin x$ ，并建立一个与之相联系的快捷菜单，用以控制曲线的线型和曲线宽度。



6.1.3 界面设计（Interface Design）

对话框是用户和计算机进行信息交流的临时窗口，它和菜单一起构成用户图形界面，对话框上有各种各样的图形对象，如图标、文本框和滚动条等，让用户进行某些操作，如设置选项或属性。

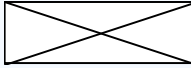
在MATLAB中，要设计一个对话框，首先要建立一个图形窗口，然后在图形窗口中使用uicontrol函数建立对话框中的控件对象。

1. 建立控件对象（ Create control object ）

MATLAB用于建立控件对象的函数是uicontrol，其调用格式：

$H = \text{uicontrol}(mH, \text{属性名1}, \text{属性值1}, \text{属性名2}, \text{属性值2} \dots)$

其中H：生成的对话框控件句柄，mH：为图形窗口句柄



2. 控件对象的属性（ **Properties of the control object** ）

（1） **Style**——定义控件对象的类型，取值可以为：

Push Button：（默认）命令按钮

Toggle Button：双位按钮

Radio Button：单选按钮

Check Box：复选框

List Box：列表框

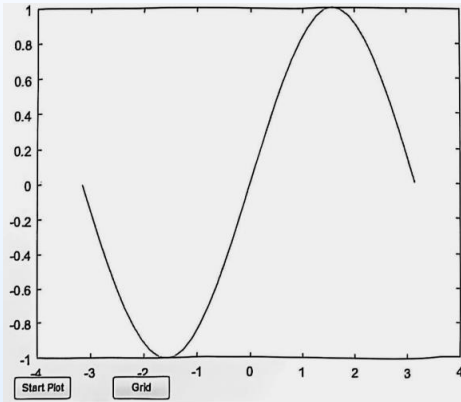
Popup Menu：弹出式菜单

Edit Box：编辑框

Static Text：静态文本框（不可编辑）

Slider：滑动条

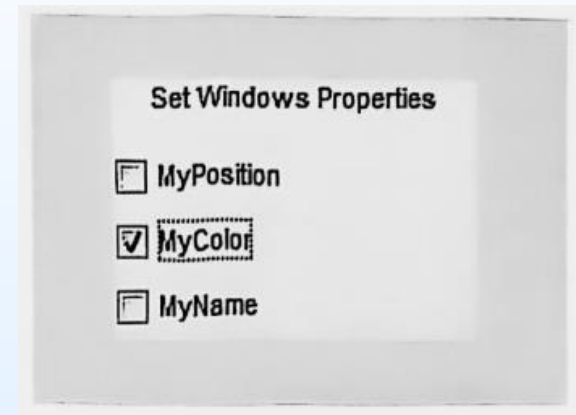
Frame：边框，用于分组或修饰界面。



按钮和双位按钮



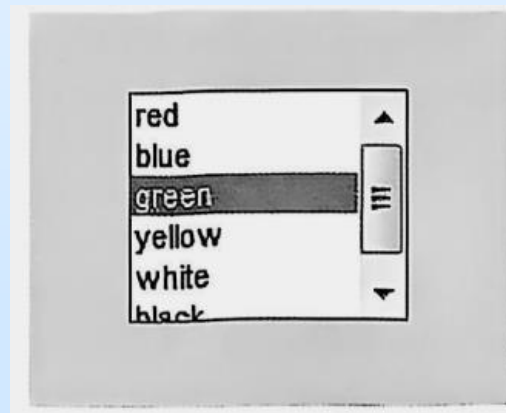
单选按钮



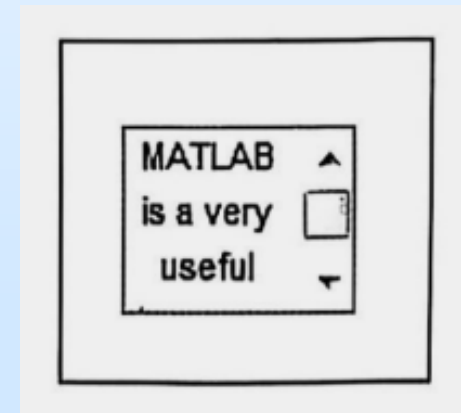
复选框



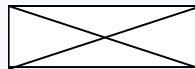
弹出框



列表框



编辑框和边框



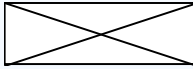
(2) **String**——属性取值是文本字符串，定义控件对象的说明文字。在按钮键、单选按钮、复选框和弹出式菜单上指定控件的说明文字。对于**可编辑文本框**，该属性设置成由用户输入的字符串。对**弹出式菜单**或**可编辑文本框**中的多个选项，每一项用垂直条(|)分隔，整个字符串用引号括起来。框架和滑标，不用引号

(3) **Callback**——MATLAB回调串，当控件激活时，回调串传给函数eval；初始值为空矩阵。

(4) **BackgroundColor**——控件背景色。3元组的RGB向量或MATLAB一个预先定义的颜色名称。缺省的背景色是浅灰色。

(5) **ForegroundColor**——控件前景（文本）色（定义控件对象说明文字的颜色）。3元素的RGB向量或MATLAB一个预先定义的颜色名称。缺省的文本色是黑色。

(6) **HorizontalAlignment**——相对于控件的文本对齐方式。
left: 左, center (默认): 居中, right: 右



(7) **Position**——位置向量[left bottom width height]。其中，[left height]表示相对于图形对象左下角的控件的左下角位置。[width height]表示控件的尺寸大小，其单位由属性Units确定。

(8) **units**——位置属性值的单位。有inches：英寸、centimeters：厘米、normalized：归一化的坐标值、points：打印设置点，等于1/72 英寸、pixels：默认，像素。

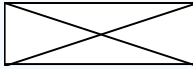
(9) **Enable**——控件使能状态
on（默认）：控件使能。激活控件将Callback字符串传给eval
off：控件不使能，标志串模糊不清。激活控件不起作用

(10) **Parent**——包含控件对象的图形句柄

(11) **Tag**——标志文本串

(12) **UserData**——用户指定的数据。

(13) **Visible**——控件对象的可视性，取值on：默认，控件对象在屏幕上可见；off：控件对象不可见，但仍然存在。



(14) **Value**——控件的当前值。文本对象和按钮不设置该属性。

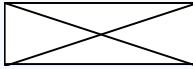
(15) **Max与Min**——属性‘Value’的最大（小）许可值，缺省值为1和0。最大（小）值取决于控件的‘Type’，当控件处于on状态时，单选按钮及复选框将Value属性值设定为Max或Min的值；该值定义了弹出式菜单最小下标值或滑标的最大值。当Max-Min>1时，可编辑文本框是多行文本。

(16) **ButtonDownFcn**——当控件被选择时，MATLAB回调串传给函数eval。初始值为空矩阵

(17) **Interruptible**——指定ButtonDownFcn和CallBack串是否可中断，取值on：默认，回调不能由其它回调中断；off：回调串可被中断

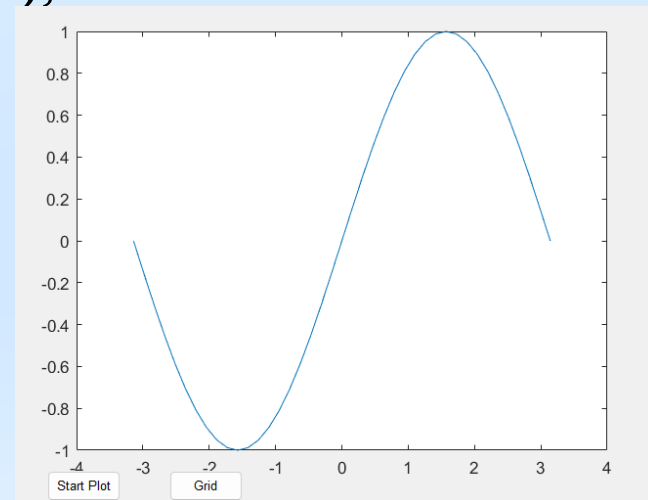
(18) **FontName、FontSize、FontAngle**等用来控制字体的属性

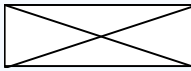
3. 建立控件对象示例 (Create a sample control object)



(1) 建立**按钮对象**，当单击该按钮时绘制出正弦曲线。同时建立双位按钮，用于控制是否给坐标加网格线。程序如下：

```
pbstart=uicontrol(gcf,'Style','push','Position',...  
[50,5,60,25],'String','Start Plot',...  
'CallBack','t=pi:pi/20:pi;plot(t,sin(t))');  
ptgrid=uicontrol(gcf,'Style','toggle','Position',...  
[150,5,60,25],'String','Grid','CallBack','grid');
```

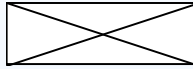




(2) 建立**单选按钮**，用来设置图形窗口的颜色，并且只能选择一种颜色。程序如下：

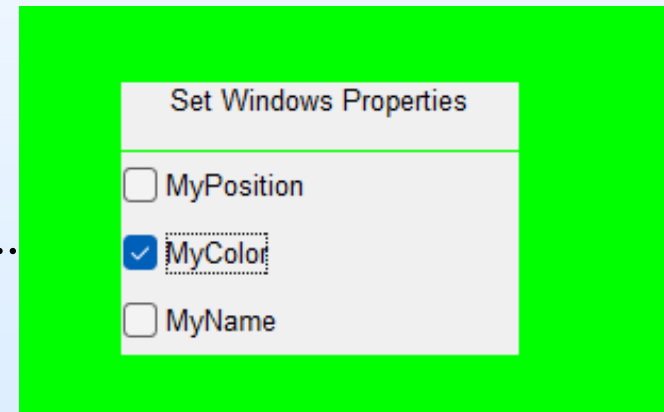
```
htxt=uicontrol(gcf,'Style','text','String',...
'Color Options','Position',[200,130,150,20]);
%建立单选按钮
hr=uicontrol(gcf,'Style','radio','String',...
'Red','Position',[200,100,150,25],'Value',1,...
'Callback',['set(hr,"Value",1);','set(hb,"Value",0);',...
'set(hy,"Value",0);','set(gcf,"Color","R")']);
hb=uicontrol(gcf,'Style','radio','String',...
'Blue','Position',[200,75,150,25],...
'Callback',['set(hb,"Value",1);','set(hr,"Value",0);',...
'set(hy,"Value",0);','set(gcf,"Color","B")']);
hy=uicontrol(gcf,'Style','radio','String',...
'Yellow','Position',[200,50,150,25],...
'Callback',['set(hy,"Value",1);','set(hr,"Value",0);',...
'set(hb,"Value",0);','set(gcf,"Color","Y")']);
```



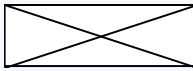


(3) 建立**复选框**，用于设置图形窗口的某些属性，如大小、颜色、标题等。程序如下：

```
htxt=uicontrol(gcf,'Style','text','Position',[200,125,150,25],...
'String','Set Windows Properties');
hp=uicontrol(gcf,'Style','check','Position',....
[200,100,150,25],'String','MyPosition',...
'Callback',['set(gcf,"Position",[10,10,300,250]);',...
'if get(hp,"Value")==1',...
'set(gcf,"Position",[10,10,600,500]);','end']]);
hc=uicontrol(gcf,'Style','check','Position',...
[200,75,150,25],'String','MyColor',...
'Callback',['set(gcf,"color","w");',...
'if get(hc,"Value")==1','set(gcf,"color","g");','end']]);
hn=uicontrol(gcf,'Style','check','Position',...
[200,50,150,25],'String','MyName',...
'Callback',['set(gcf,"Name","复选框未选中");',...
'if get(hn,"Value")==1',...
'set(gcf,"Name","复选框被选中");','end']]);
```

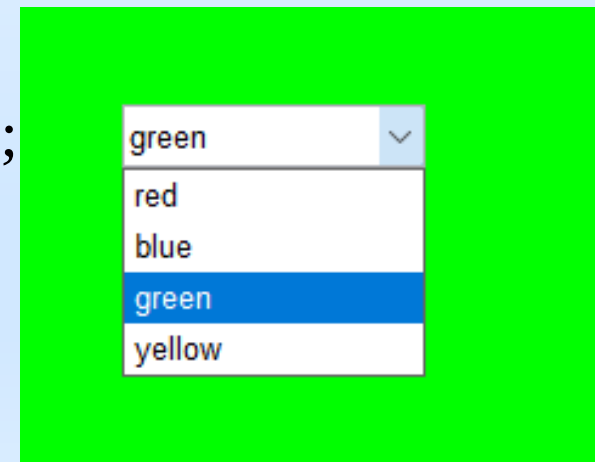


(4) 建立**弹出框**，其列表中包含一组可供选择的颜色。当选择某种颜色时，就将图形窗口的背景色设置为该颜色。

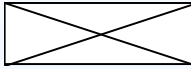


弹出框可选项在String 属性中设置，每项之间用竖线字符“|”隔开，并用单撇号将所有的选项括起来。Value属性的值是弹出式列表中选项的序号。例如，如果用户选择列表中的第2项,那么Value的属性值就是2。程序如下：

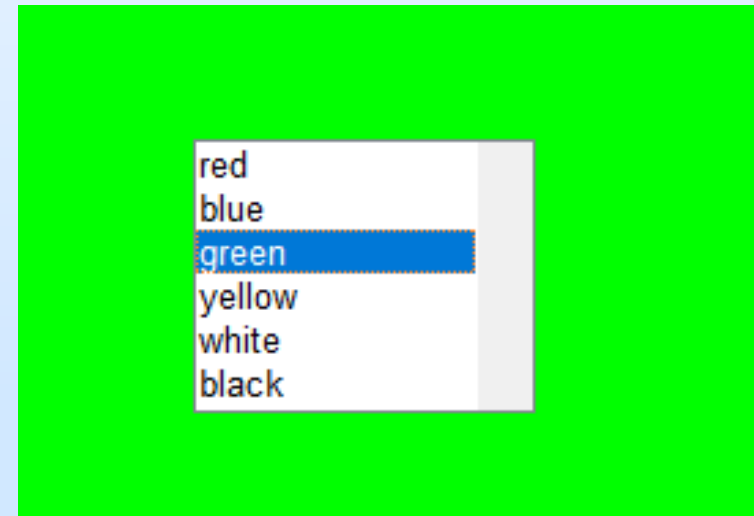
```
hpop=uicontrol(gcf,'Style','popup','String',...  
'red|blue|green|yellow','Position',[100,100,100,80],...  
'Callback',['cbcol=["R","B","G","Y"];',...  
'set(gcf,"Color",cbcol(get(hpop,"Value")))]);
```



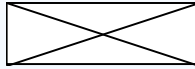
(5) 建立列表框，其作用与（4）相同。程序如下：



```
hl=uicontrol(gcf,'Style','list',...  
'String','red|blue|green|yellow|white|black',...  
'Position',[100,100,100,80],'Callback',...  
['cbcol=["r","b","g","y","w","k"];',...  
'set(gcf,"color",cbcol(get(hl,"Value")));']);
```



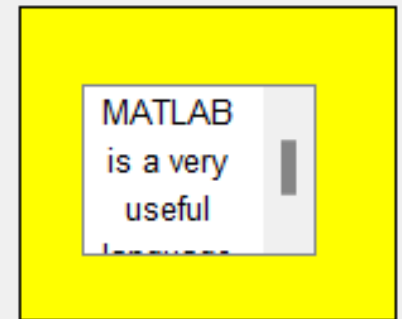
(6) 建立一个编辑框，并加边框。

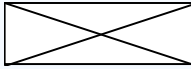


当需要用边框组织控件对象时，必须在定义控件对象之前建立边框对象，或者说边框对象必须覆盖该组中所有的控件对象。为了留出边框的边界，边框对象所占用的区域至少要比该组中所有控件对象占用的区域大。

编辑框分多行编辑框与单行编辑框。属性 **Max**与属性 **Min**之差小于或等于1时，为单行编辑框，否则为多行编辑框。程序如下：

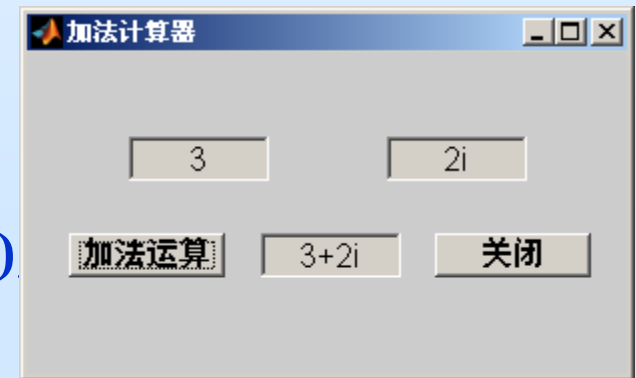
```
ftdir=uicontrol(gcf,'Style','frame',...  
'back','y','Position',[30,180,120,100]);  
edmulti=uicontrol(gcf,'Style','edit ',...  
'String','MATLAB is a very useful language.',...  
'Position',[50,200,75,55],'Max',2,'back','w');
```

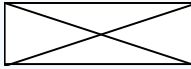




Example 9-12 设计一个如下图所示的加法计算器

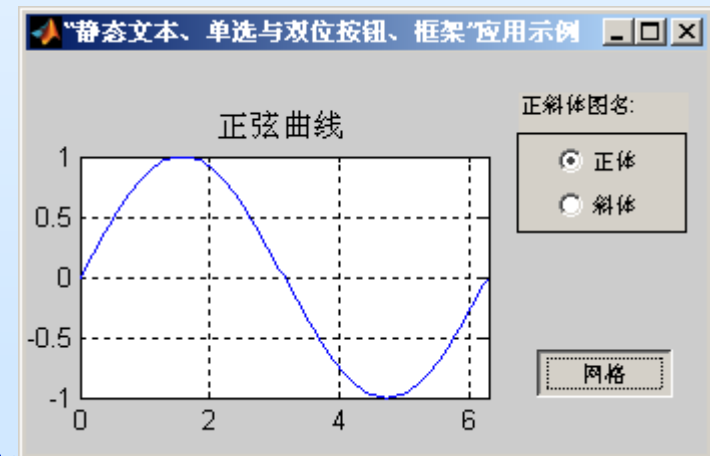
```
hf=figure('Position',[50,50,300,160],...
    'Name','加法计算器','NumberTitle','off','MenuBar','none');
set(hf,'defaultuicontrolunits','normalized','defaultuicontrolfontsize',12);
hedit1=uicontrol(hf,'style','edit','position',[0.17,0.6,0.232,0.14],...
    'callback','z1=str2num(get(gcbo,'string'))');
hedit2=uicontrol(hf,'style','edit','position',[0.6,0.6,0.232,0.14],...
    'callback','z2=str2num(get(gcbo,'string'))');
hedit3=uicontrol(hf,'style','edit','position',[0.39,0.3,0.232,0.14]);
ht1=uicontrol(hf,'style','push',...
    'string','加法运算','fontname','黑体',...
    'position',[0.07,0.3,0.26,0.14],...
    'callback','set(hedit3,'string',num2str(z1+z2))');
ht2=uicontrol(hf,'style','push',...
    'string','关闭','fontname','黑体',...
    'position',[0.68,0.3,0.26,0.14],'callback','close(hf)');
```





Example 9-13 创建一个界面包含4种控件：静态文本、单选按钮、双位按钮、框架。

```
clf('reset') %初始化图像窗口
set(gcf,'Name','“静态文本、单选与双位按钮、框架” 应用示例',
'numbertitle','off', 'menubar','none')
set(gcf,'position',[50,50,340,200]);
set(gcf,'defaultuicontrolunits','normalized') %设置缺省控件单位
h_axes=axes('position',[0.08,0.14,0.6,0.6]);
t=0:pi/50:2*pi;y=sin(t);plot(t,y);
set(h_axes,'xlim',[0,2*pi]);
htitle=title('正弦曲线');
%创建静态文本框
uicontrol('style','text','string','正斜体图名:',...
'position',[0.723,0.8,0.25,0.1],'horizontal','left');
%创建框架
uicontrol('style','frame','position',[0.72,0.55,0.25,0.25]);
```



%创建单选按钮

```
hr1=uicontrol(gcf,'style','radio','string','正体',...
'position',[0.78,0.69,0.18,0.08]);
```

```
set(hr1,'value',get(hr1,'Max'));%先让hr1处于激活态，默认的Max为1
```

```
set(hr1,'callback',['set(hr1,'value',get(hr1,'max'))'],...
'set(hr2,'value',get(hr2,'min'))','set(htitle,'fontangle','normal')]);
```

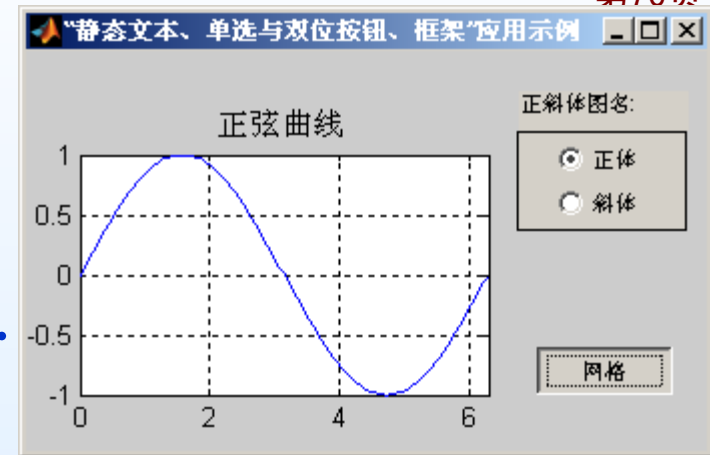
```
hr2=uicontrol(gcf,'style','radio','string','斜体',...
```

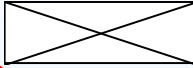
```
'position',[0.78,0.58,0.18,0.08],'callback',['set(hr1,'value',get(hr1,'min'))'],...
'set(hr2,'value',get(hr2,'max'))','set(htitle,'fontangle','italic')]);
```

%创建双位按钮

```
ht=uicontrol(gcf,'style','toggle','string','网格',...
```

```
'position',[0.75,0.14,0.2,0.12],'callback','grid');
```





9.2图形用户界面开发环境（GUIDE）

为了更方便简洁地进行界面设计，MATLAB提供了一个可视化图形界面开发环境GUIDE（Graphical User Interface Development Environment）。这种开发环境如同VB、VC++等程序，设计用户交互界面的过程就是把需要的控件从控件模板拖或复制到界面编辑区，直观、方便，实现了“所见即所得”的可视化设计。

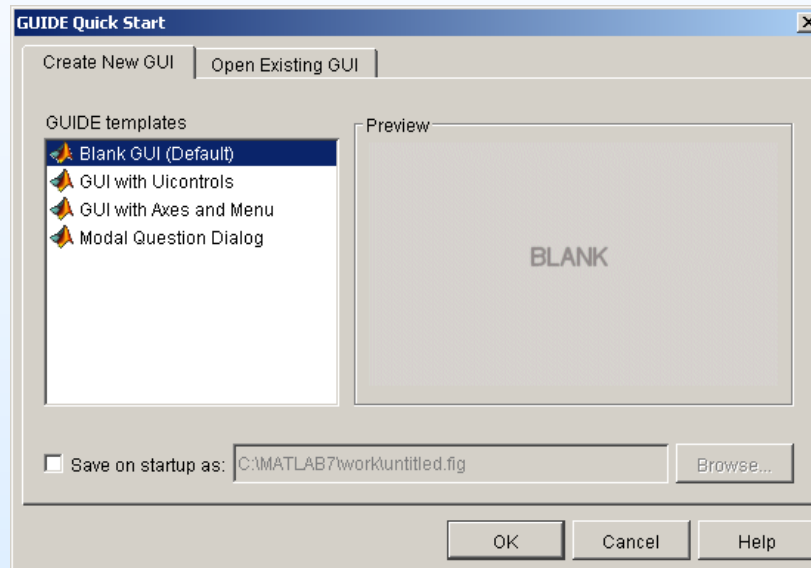
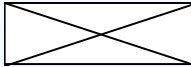
9.2.1 GUIDE窗口（Window）

用户要进入图形用户界面设计窗口，可以在MATLAB环境下：

选择File→New → GUI子菜单

或 在命令窗口键入： `guide`

即可得到下图：

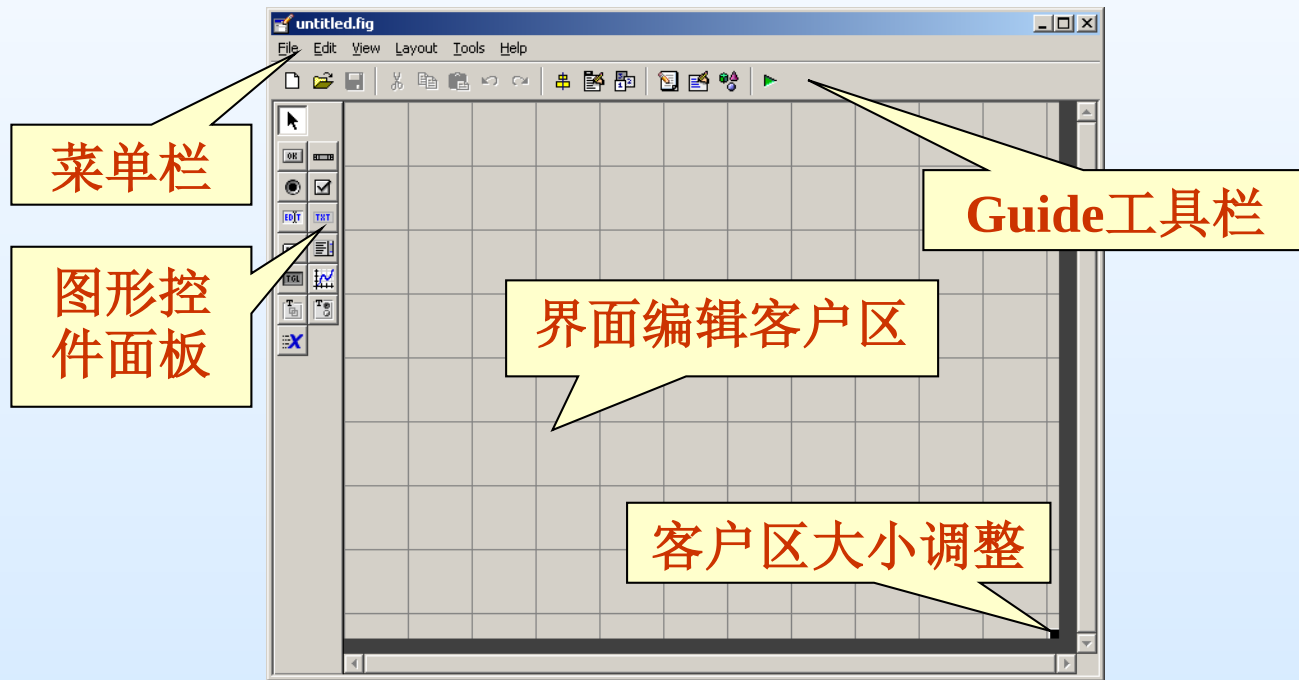


MATLAB为GUI设计一共准备了4种模板，分别是Blank GUI(默认)、GUI with Uicontrols(带控件对象的GUI模板)、GUI with Axes and Menu(带坐标轴与菜单的GUI模板)与Modal Question Dialog(带模式问话对话框的GUI模板)。

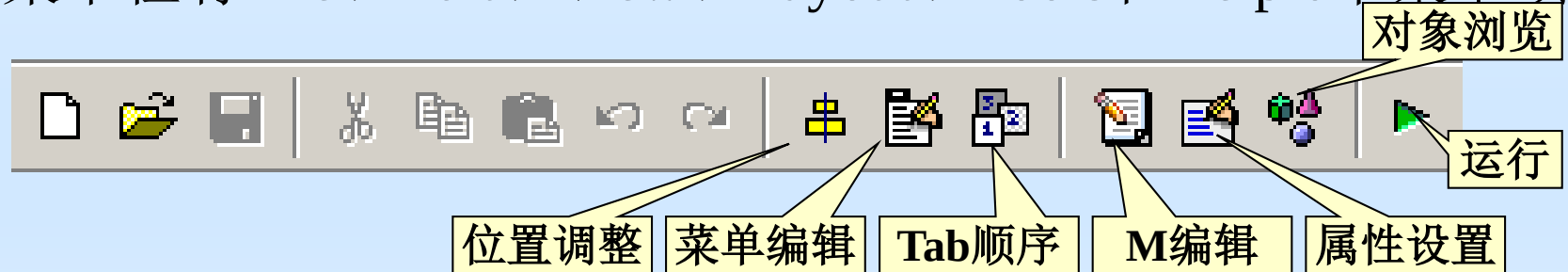
当用户选择不同的模板时，在GUI设计模板界面的右边就会显示出与该模板对应的GUI图形。

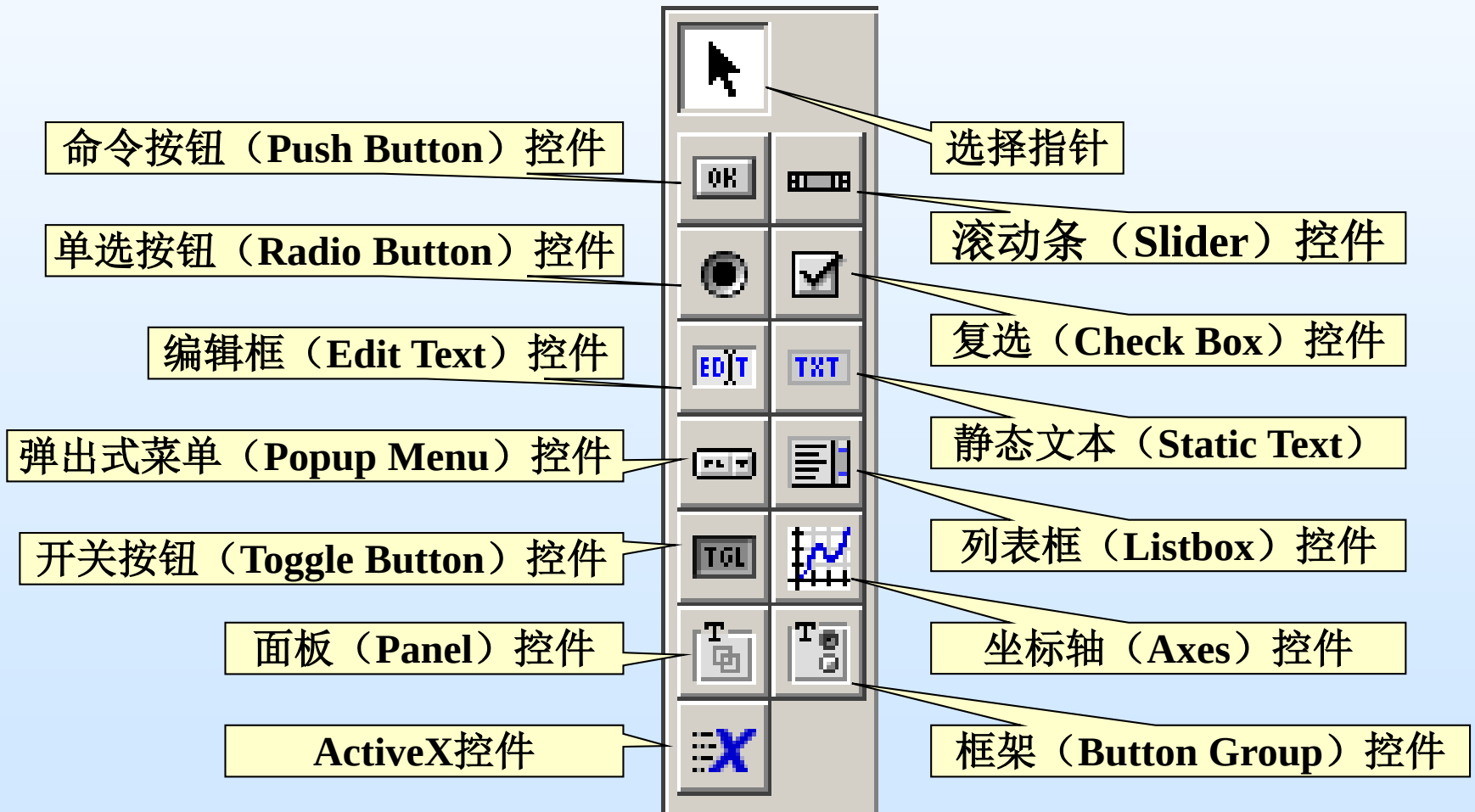
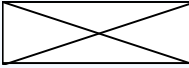
单击OK后，便可以进入GUIDE设计主窗口。

1. 窗口组成 (Window Composition)

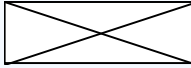


菜单栏有File、Edit、View、Layout、Tools和Help 6个菜单项





面板控件 (Panel Control)

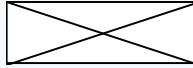


2. 基本操作（ Basic Operation ）

为了添加控件，可以从GUI设计窗口的控件面板中选择一个对象，然后以拖曳方式在客户区建立该对象。在GUI设计窗口创建对象后，通过双击该对象，就会显示该对象的属性查看器，通过它可以设置该对象的属性值。

在选中对象的前提下，单击鼠标右键，会弹出一个快捷菜单，可以从中选择某个子菜单进行相应的操作。在对象设计区右击鼠标，会显示与图形窗口有关的快捷菜单。

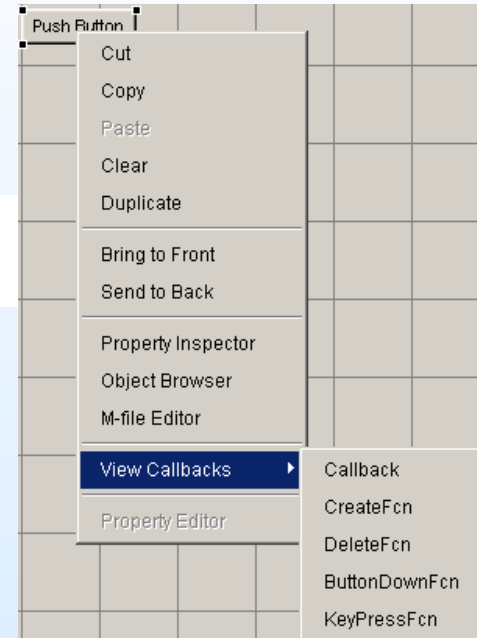
开发好一个程序的过程主要有两个重要的内容：**布局好图形用户界面控件**和**给这个用户界面编写代码**。GUIDE环境把建立好的GUI文件保存为两个伴随文件：**FIG文件**（保存用户界面设计的代码）和**M文件**（保存着GUI进行调用的各种函数和代码）。

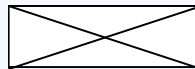


6.2.2 GUIDE常用工具 (Common Tools)

1. 布局编辑器(Layout Editor)

- **Cut:** 对选中的控件进行剪切操作。
- **Copy:** 复制选中的控件。
- **Paste:** 粘贴复制的控件。
- **Delete:** 删除选中的控件。
- **Duplicate:** 对选中的控件进行复制并粘贴。
- **Property Inspector:** 对选中的控件打开属性编辑窗口。
- **Object Browser:** 打开对象浏览窗口。
- **ButtonDownFcn:** 按下鼠标时控件回调的函数。
- **Callback:** 单击鼠标时控件回调的函数或功能。
- **CreateFcn:** 定义控件在创建阶段执行的回调例程。
- **DeleteFcn:** 定义在对象的删除阶段执行的回调例程。





2. 属性编辑器(Property Inspector)

可查看每个对象的属性值，也可修改设置对象的属性值。

(1) 控件风格和外观

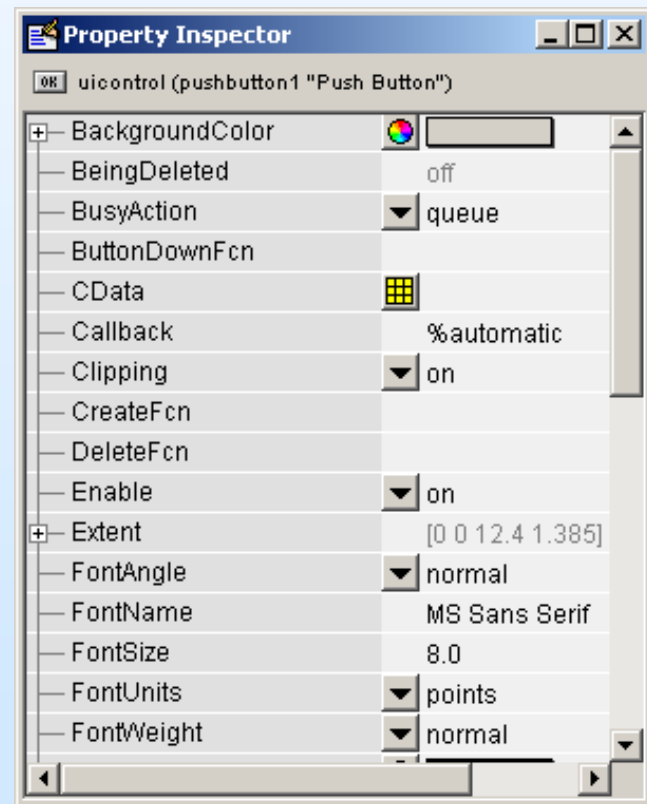
➤ **BackgroundColor:** 设置控件背景颜色，使用[R G B]或颜色定义。

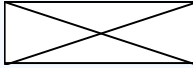
➤ **CData:** 在控件上显示的真彩色图像，使用矩阵表示。

➤ **ForegroundColor:** 文本颜色。

➤ **String**属性：控件上的文本，以及列表框和弹出菜单的选项。

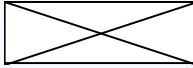
➤ **Visible:** 控件是否可见。





(2) 对象的常规信息 (General Information about the Object)

- **Enable:** 表示此控件的使能状态，设置为“on”，表示可选，为“off”时则表示不可选。
- **Style:** 控件对象类型。
- **Tag:** 控件表示（用户定义）。
- **TooltipString:** 提示信息显示。当鼠标指针位于此控件上时，显示提示信息。
- **UserData:** 用户指定数据。
- **Position:** 控件对象的尺寸和位置。
- **Units:** 设置控件的位置及大小的单位
- 有关字体的属性，如 **FontAngle**、**FontName**等。

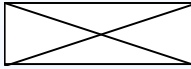


(3) 控件回调函数的执行 (the Execution of Control Callback Function)

- BusyAction: 处理回调函数的中断。有两种选项: 即Cancel: 取消中断事件, queue: 排队 (默认设置)。
- ButtonDownFcn属性: 按钮按下时的处理函数。
- CallBack属性: 是连接程序界面整个程序系统的实质性功能的纽带。该属性值应该为一个可以直接求值的字符串, 在该对象被选中和改变时, 系统将自动地对字符串进行求值。
- CreateFcn: 在对象产生过程中执行的回调函数。
- DeleteFcn: 删除对象过程中执行的回调函数。
- Interruptible属性: 指定当前的回调函数在执行时是否允许中断, 去执行其他的函数。

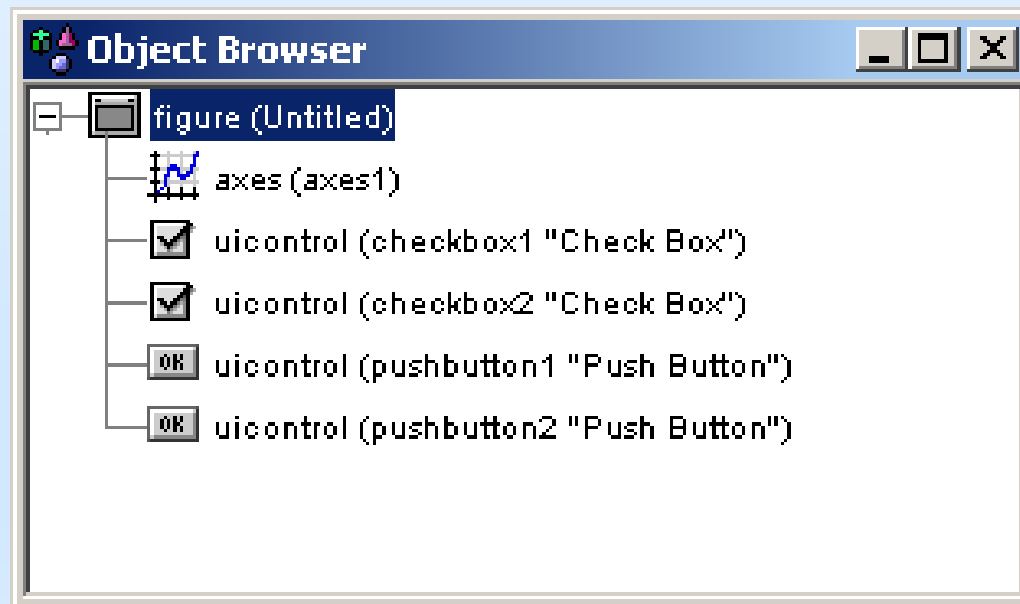
(4) 控件当前状态信息 (Current Status Information of Control)

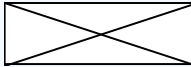
- ListboxTop: 在列表框中显示的最顶层的字符串的索引。
- Max (Min) : 最大 (小) 值。
- Value: 控件的当前值。



3. 对象浏览器(Object Browser)

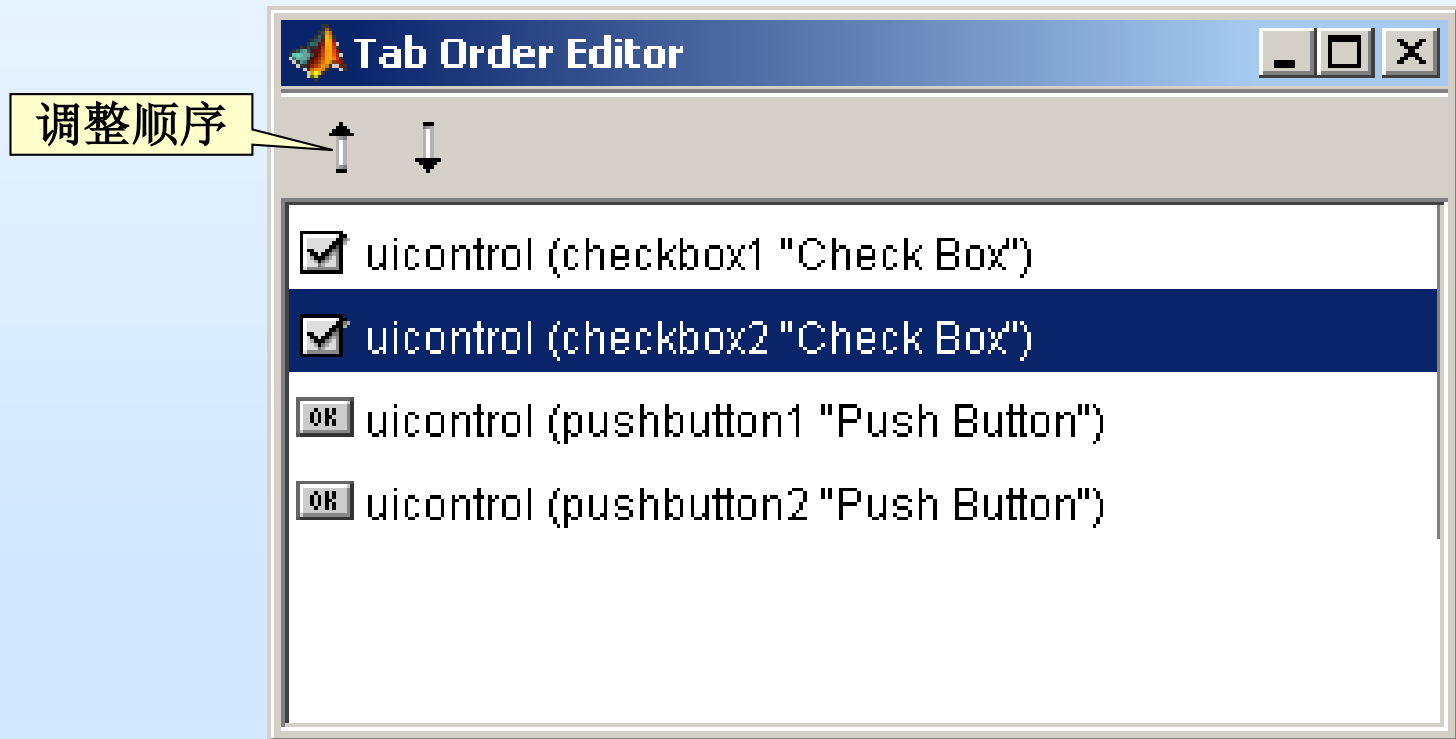
对象浏览器里面列出了所有对象的树状结构，这些结构就是当前正在设计的GUI程序中用到的所有对象。利用它可观察当前设计阶段的各个句柄图形对象，也可以单击该对象以打开相应的属性编辑器。

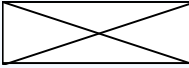




4. Tab顺序编辑器(Tab Order Editor)

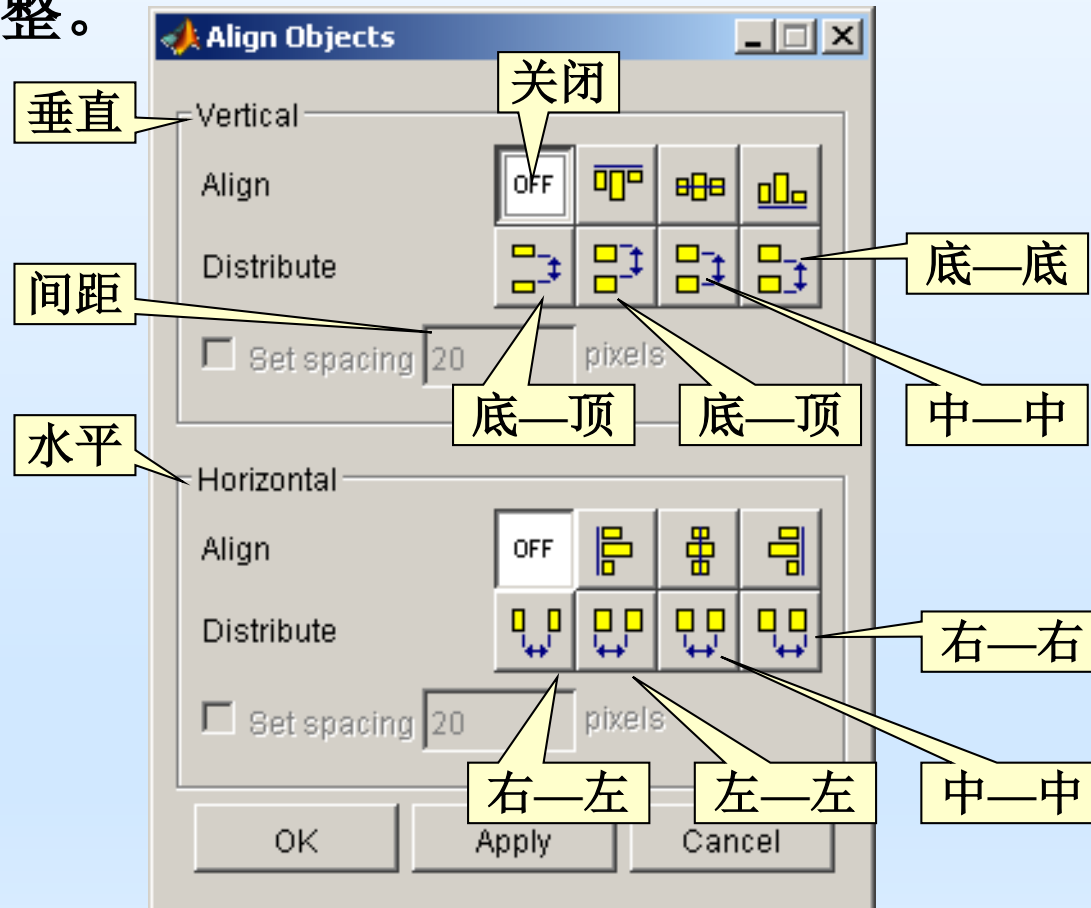
通过该工具，设置当按下键盘上的Tab键时，对象被选中的先后顺序。

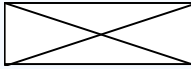




5. 对齐工具(Alignment Tool)

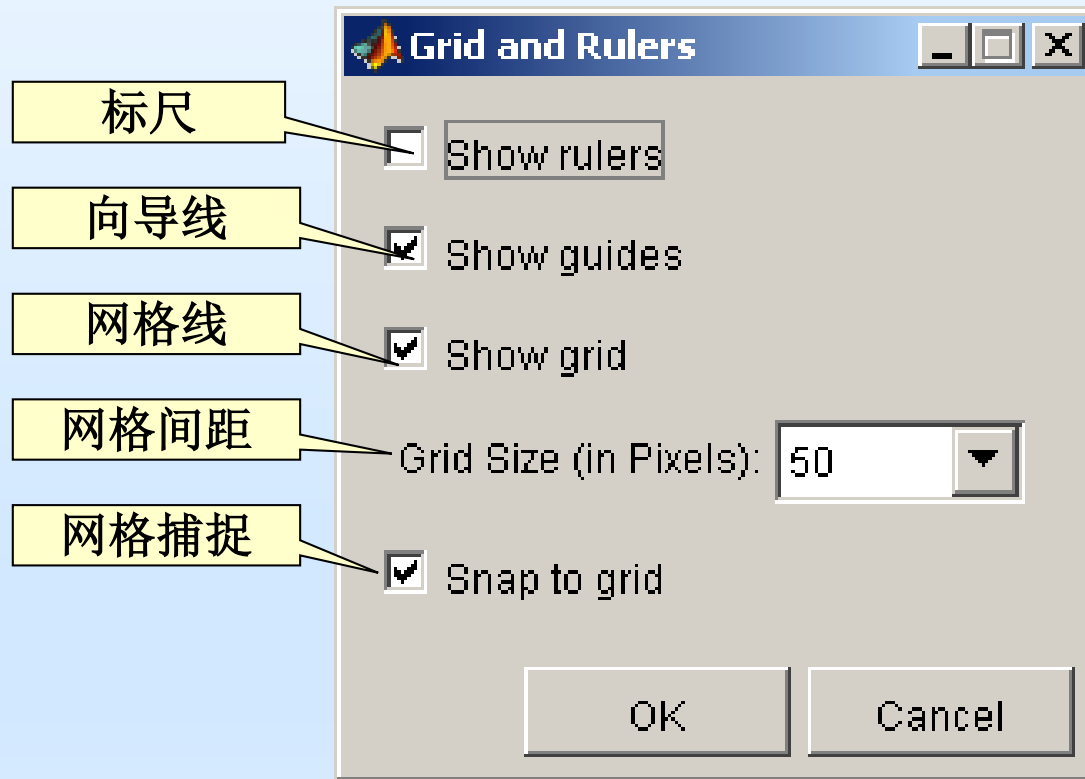
利用该工具可水平左右、垂直上下对多个对象的位置进行调整。

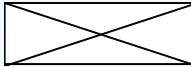




6. 网格标尺设置编辑器(Grid and Rulers)

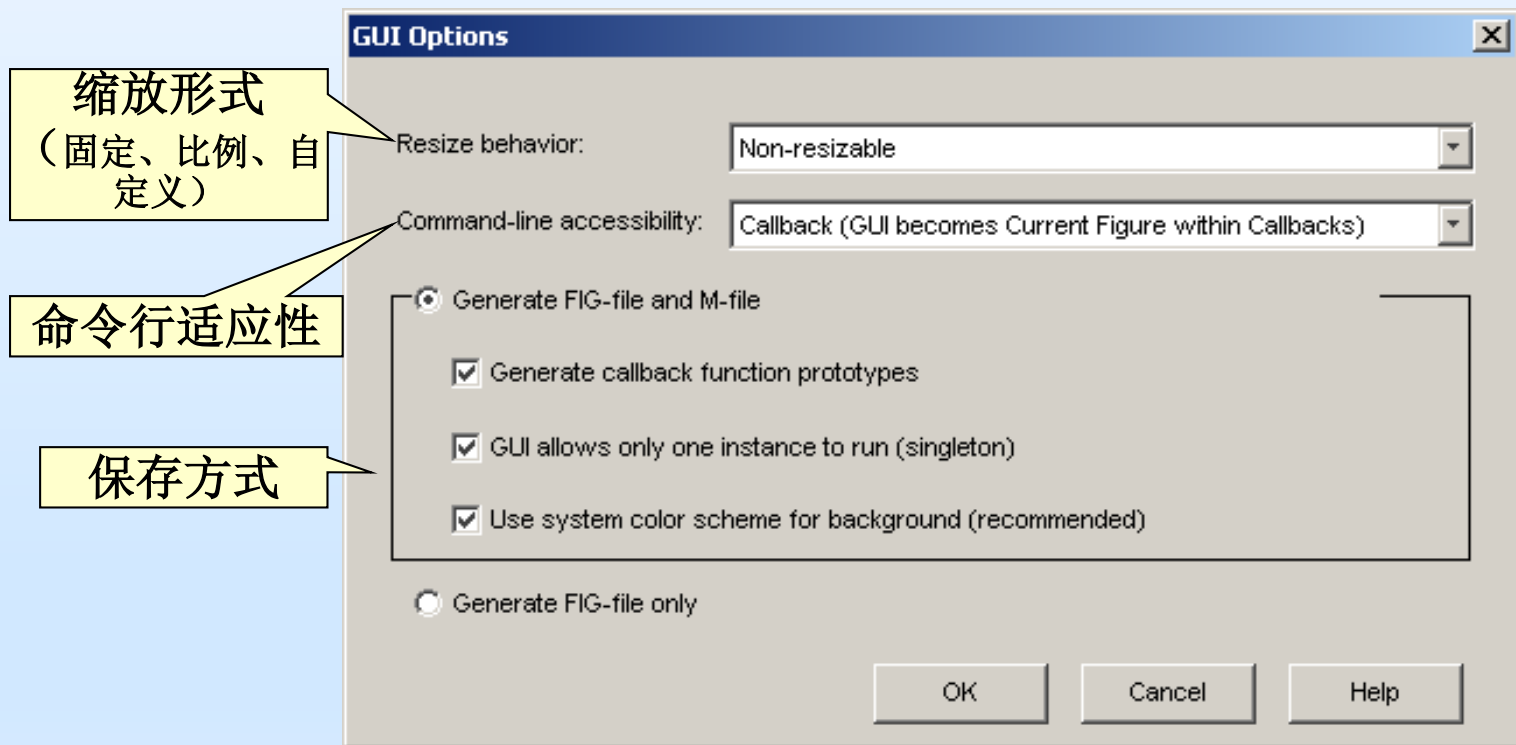
为了方便用户的界面设计，可以通过菜单**Tools**→**Grid and Rulers**在GUI界面中添加网格与标尺。

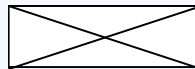




7. GUI应用属性设置编辑器(GUI Options)

为了方便用户的界面设计，可以通过菜单Tools→GUI Options或快捷菜单GUI Option设置GUI应用属性。

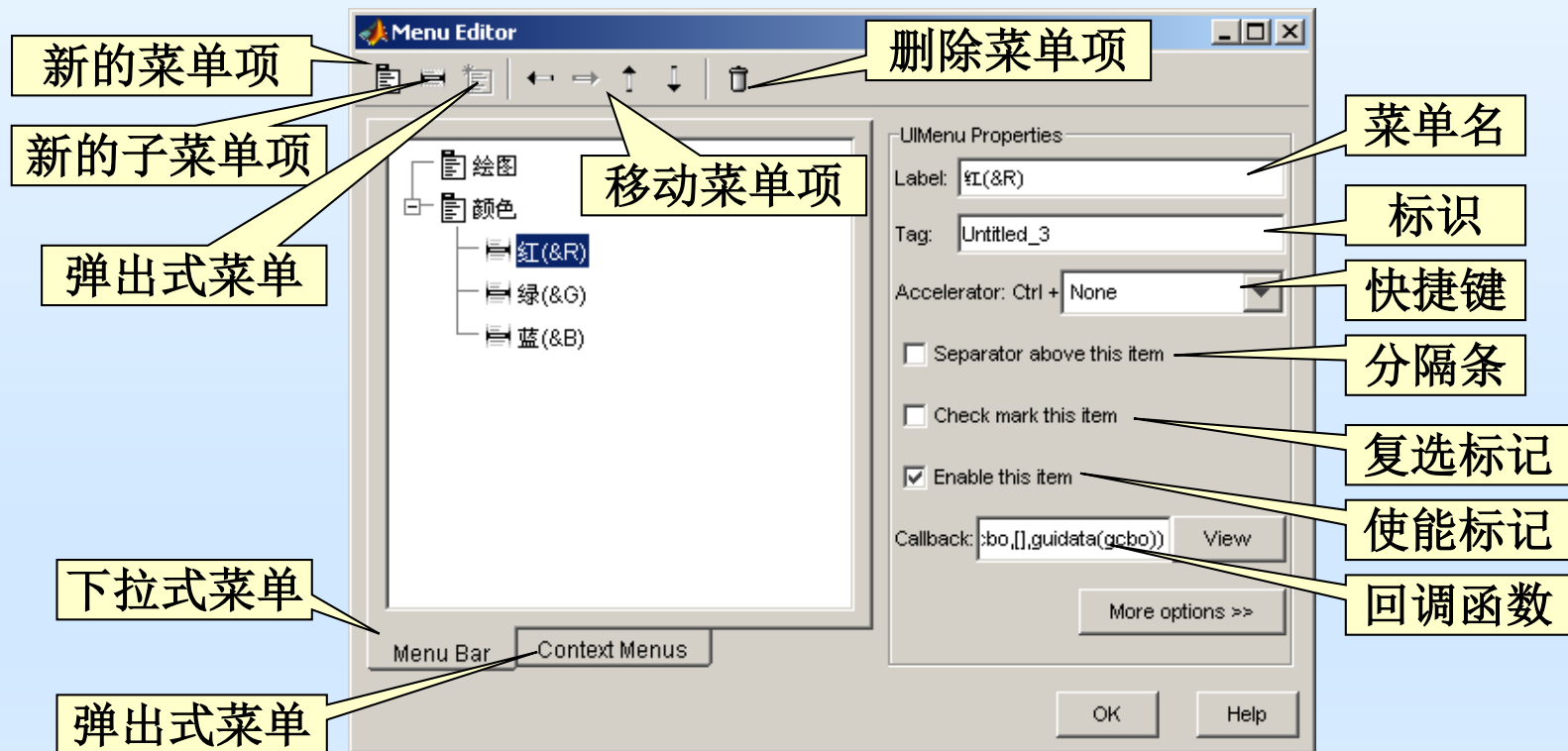


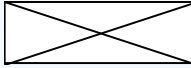


8. 菜单编辑器(Menu Editor)

菜单编辑器提供了两种菜单类型的编辑功能：

- 下拉式菜单 (Menu Bar)
- 弹出式菜单 (Context Menu)





6.2.3 GUIDE应用示例

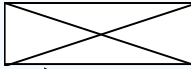
使用Guide进行GUI的界面设计，就像在纸上绘制界面一样简单。一般来说，具体的GUI开发过程可分为5个步骤：**GUI界面的设计和布局、GUI的编程、菜单的设计和布局、菜单的编程以及调试运行并保存。**

当界面设计好并进行保存后，系统将生成.fig与.m文件，其中的M文件由以下几部分构成：

- 1) 系统初始化文件，其中包括GUI设计
- 2) 各种控件的响应函数，其中的回调函数(Tag_Callback)为空，用户需插入相应的代码。

在编写调用的函数文件中，**handles变量**非常重要，实际上它是一个结构数组，它存储所有在图形界面中的控件、菜单和轴对象的句柄，而每个对象的句柄被存储在handles结构数组的一个字段中，名称与对象的Tag属性相同。

例如：`handles.figure1`

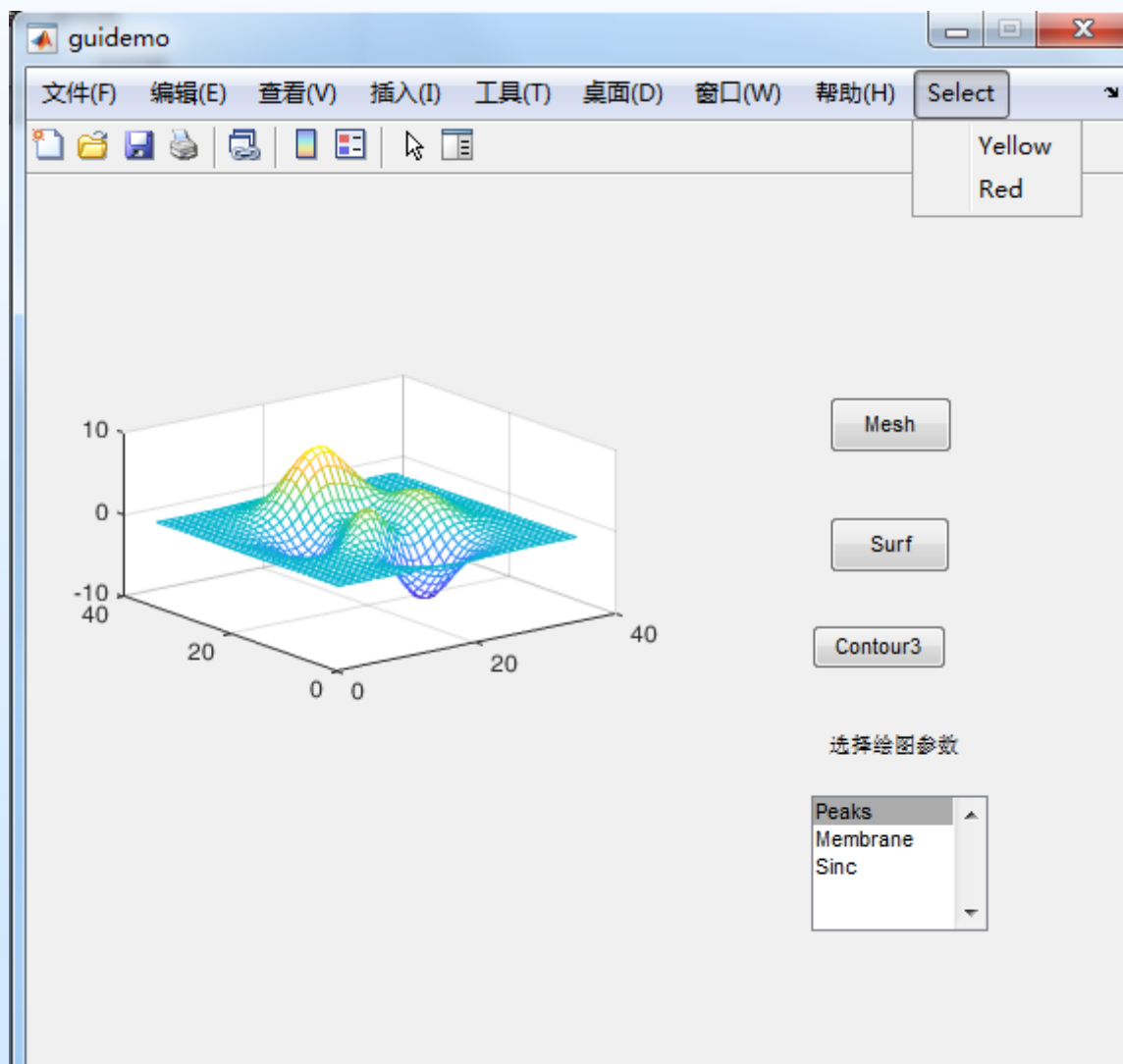
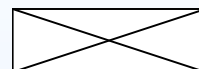


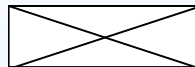
如果需要在函数间传递数据，可先给handles结构数组建立一个字段，然后调用guidata函数产生新的handles结构并存储数据。例如：`handle.x_dat=x;`

`guidata(hObject,handles);`

当再次读取数据时，只需输入：`x=handle.x_dat`

Example 9-14 利用GUI设计工具设计如下图所示的用户界面。界面包括一个用于显示图形的坐标轴对象，显示的图形包括表面图、网格图和等高线图。绘图图形的功能通过3个命令按钮来实现，用户通过单击相应的按钮，即可绘制相应图形。绘制图形所需要的数据通过一个弹出框来选取。在弹出框中包括3个选项，分别对应MATLAB的数据函数Peaks、Membrane和用户自定义的绘图数据Sinc。在图形窗口默认的菜单条上添加一个菜单项Select，下设两个子菜单Yellow和Red以调整窗口的颜色。





- (1) 打开图形用户界面设计窗口，添加有关图形对象。
- (2) 利用属性检查器设置界面对象的属性。
- (3) 利用对齐对象工具，调整控件位置。
- (4) 保存图形用户界面。（设计的图形界面保存为 **guidemo.fig** 文件，系统还自动生成一个 **guidemo.m** 文件，用于保存图形对象的程序代码）

- (5) 编写代码，实现控件功能。

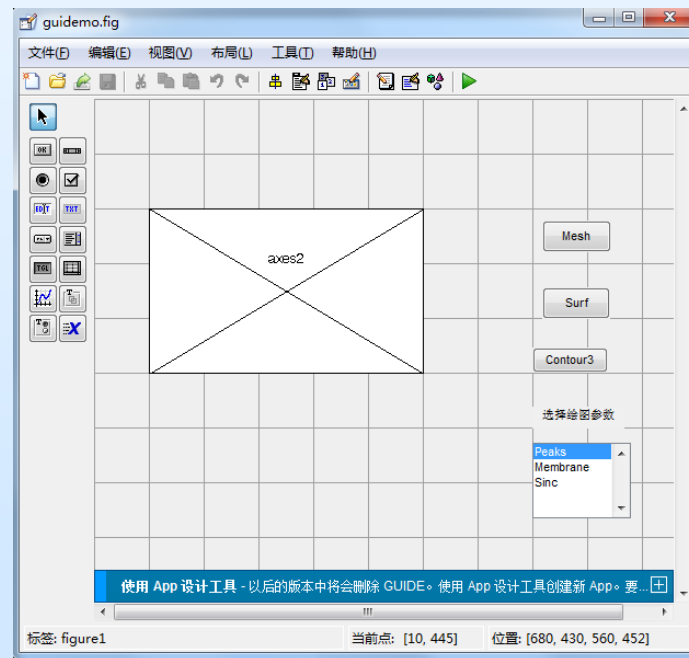
- ① 打开图形窗口事件编写响应代码；
- ② 列表框编写响应代码；
- ③ **Mesh**按钮编写响应代码；
- ④ **Surf**按钮编写响应代码；
- ⑤ **Contour3**按钮编写响应代码。

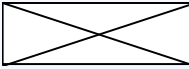
- (6) 添加Select菜单。

标签为**Yellow**的子菜单的回调属性设为`set(gcf, 'Color','y');`

标签为**Red**的子菜单的回调属性设为`set(gcf, 'Color','r');`

- (7) 运行图形用户界面。





- 打开图形窗口事件编写响应代码（guidemo_OpeningFcn函数）

%varargin开头的注解语句下输入如下代码：

```
handles.peaks=peaks(35);
```

```
handles.membrane=membrane;
```

```
[x,y]=meshgrid(-8:0.3:8);
```

```
r=sqrt(x.^2+y.^2);
```

```
sinc=sin(r)./(r+eps);
```

```
handles.sinc=sinc;
```

```
handles.current_data=handles.membrane;%当前的图像，没有任何操作时
```

```
surf(handles.current_data)
```

- Mesh按钮编写响应代码

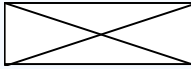
```
mesh(handles.current_data)
```

- Surf按钮编写响应代码

```
surf(handles.current_data)
```

- Contour3按钮编写响应代码

```
contour3(handles.current_data)
```



- 列表框编写响应代码（ChooseFun_Callback函数）

%handles开头的注解语句下输入如下代码：

```
str=get(hObject,'String');
```

```
val=get(hObject,'Value');
```

```
%设置用所选函数产生当前数据集
```

```
switch str{val}
```

```
    case 'Peaks'
```

```
        handles.current_data=handles.peaks;
```

```
    case 'Membrane'
```

```
        handles.current_data=handles.membrane;
```

```
    case 'Sinc'
```

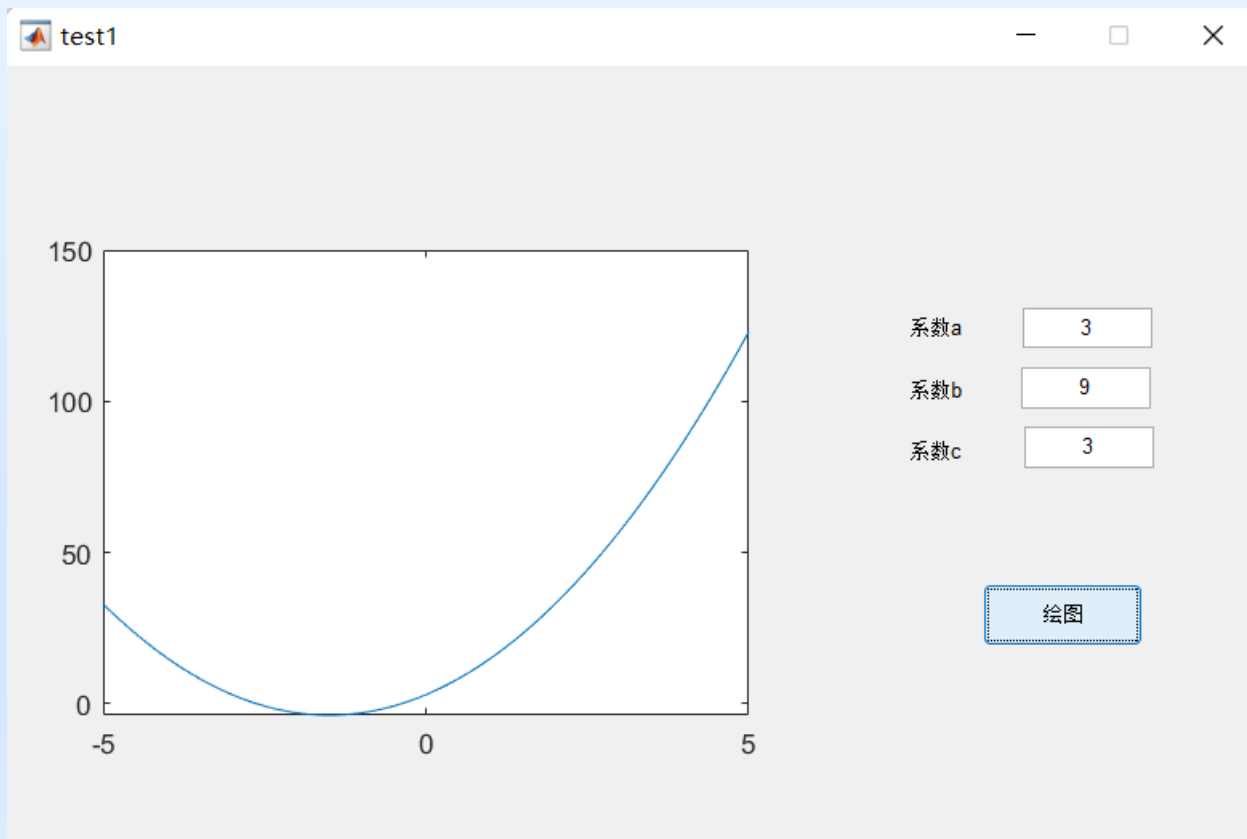
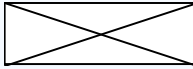
```
        handles.current_data=handles.sinc;
```

```
end
```

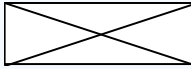
```
%保存句柄结构
```

```
guidata(hObject,handles)
```


Example 9-15 利用GUI设计工具设计如下图所示的用户界面。在编辑框输入a、b、c的值，当单击“绘图”按钮时，绘制 ax^2+bx+c 的图像，观察参数a、b、c对图像的影响。



操作步骤:



(1) 新建一个GUI，在图像窗口上添加1个坐标轴、3个静态文本框、3个编辑框和1个按钮。图形窗口的MenuBar属性设置为none，Name属性设置为“图形演示”，其余控件的属性按图设置。

(2) 在按钮的pushbutton1_Callback函数中添加如下代码：

```
function pushbutton1_Callback(hObject, eventdata, handles)
% hObject handle to pushbutton2 (see GCBO)
% eventdata reserved - to be defined in a future version of MATLAB
% handles structure with handles and user data (see GUIDATA)
com1=get(handles.edit1,'String');
com2=get(handles.edit2,'String');
com3=get(handles.edit3,'String');
a=eval(com1); b=eval(com2); c=eval(com3);
x=-5:0.1:5; y=a*x.*x+b*x+c;
axis([-5 5 -20 20]);
plot(x,y)
hold on
```